UNIVERSIDAD PANAMERICANA

Facultad De Ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación

Programación V/ Base de datos 2

Proyecto HIPSTAGRAM – Fase 2

(DERCAS)

Integrantes:

Douglas Esaú Catú Otzoy (000140060)

Kevin Eliazar Herrera Alburez (000113637)

Priscila Andrea Güitz del Cid (000139626)

Chimaltenango, Chimaltenango, abril 2026

Índice

1. Antecedentes 2
2. Objetivos 3
2.1 Objetivo General 3
2.2 Objetivos Específicos 3
3. Descripción de Procesos 4
3.1 Proceso de Registro e Inicio de Sesión 4
3.2 Proceso de Publicación de Fotografía 4
3.3 Proceso de Votación (Like/Dislike) 4
3.4 Proceso de Búsqueda 4
3.5 Proceso de Moderación Administrativa 4
4. Diagramas de bloque de procesos 5
5. Bloques 7
5.1 Procesos 10
5.1.1 Proceso – Login 10
5.1.2 Proceso – Publicar Foto 11
5.1.3. Proceso – Votar (Like/Dislike) 12
5.1.4. Proceso – Moderación (Admin) 13
5.1.5. Proceso – Búsqueda 14
6. Descripción de procedimientos 15
6.1 Registro de Usuario 15
6.2 Inicio de Sesión 16
6.3 Publicación de Fotografía 16
6.4 Votación (Like/Dislike) 17
6.5 Creación de Comentario 18
6.6 Búsqueda de Contenido 18
6.7 Moderación Administrativa 19
7. Diagramas de flujos del proceso 20
7.1 Diagrama De Flujo – Registro De Usuario 20
7.2. Diagrama De Flujo – Login 22
7.3 Diagrama De Flujo – Publicar Foto 24

7.4 Diagrama De Flujo – Votar (Like / Dislike) 26
7.5 Diagrama De Flujo – Eliminar Publicación 28
7.6 Diagrama De Flujo – Búsqueda..... 30
8. Listado de requerimientos funcionales de la aplicación...... 32
8.1 Requerimientos Funcionales**..... 32**
9. Definición del alcance..... 33
9.1 Apartado del alcance para el rol de usuario..... 33
10. Identificación de módulos que integran la aplicación. 34
10.1 Módulo API Gateway 34
10.2 Módulo de Autenticación de Usuarios 34
10.3 Módulo para Publicaciones 34
10.4 Módulo de Administración 35
10.5 Módulo de Búsqueda 35
11. Diagrama de componentes 37
12. Definición de requerimientos no funcionales 38
12.1 Seguridad. 38
12.2 Validación. 38
12.3 Rendimiento. 38
12.4 Disponibilidad. 38
13. Diagramas de casos de uso..... 39
13.1 Descripción de los Diagramas de Flujo del Sistema 39
14. Diagramas de flujo de datos 41
15. Prototipos e interfaces de los requerimientos funcionales 42
15.1 Prototipo en Figma 42
15.2 Paleta de Colores del Sistema de Diseño..... 43
15.3 Mapa de Navegación del Sistema..... 43
16. Recursos tecnológicos a utilizar (listado con costos) 56
16.1 Tecnologías de desarrollo. 56
16.2 Tecnologías de persistencia y almacenamiento. 56
16.3 Tecnologías de infraestructura. 56
17. Prioridad de los requerimientos a desarrollar..... 57

17.1. Alta prioridad. 57
17.2. Prioridad media. 57
17.3 Baja prioridad. 57

DOCUMENTO DE ESPECIFICACIÓN DE REQUERIMIENTOS

(DERCAS)

Proyecto HIPSTAGRAM

Plataforma de Red Social con Microservicios

Versión	2.0
Fecha	Abril 2026
Equipo	Equipo de Desarrollo — 3 integrantes
Curso	Proyecto General — Ing. Mario Obed Morales Guitz

1. Antecedentes

En el panorama tecnológico contemporáneo, las plataformas de interacción social a gran escala han consolidado paradigmas altamente exigentes para la distribución de contenido multimedia, la indexación dinámica mediante metadatos (*hashtags*) y la gobernanza a través de políticas de moderación automatizadas. Sin embargo, comprender y replicar la infraestructura subyacente de estos ecosistemas representa un desafío de ingeniería complejo. La magnitud de estas operaciones trasciende la simple transferencia de archivos binarios; exige la orquestación eficiente de la persistencia transaccional, la implementación de perímetros de seguridad de confianza cero (*Zero Trust*) y el diseño de topologías elásticas capaces de sostener alta disponibilidad (HA) frente a picos de concurrencia masiva.

En respuesta a esta demanda de rigor analítico y técnico, surge Hipstagram: una plataforma distribuida orientada a la publicación, interacción y descubrimiento de material fotográfico. A diferencia de las soluciones comerciales, el valor central de este artefacto radica en su enfoque pedagógico y estructural. La arquitectura del sistema abandona los modelos monolíticos tradicionales para fundamentarse en un patrón de microservicios altamente desacoplados, los cuales son contenerizados mediante el ecosistema Docker y aprovisionados en la infraestructura de computación en la nube de Amazon Web Services (AWS).

La conceptualización y construcción de este proyecto constituye un entorno de validación empírica para consolidar las competencias transversales de la ingeniería de software. Su ciclo de desarrollo articula de manera práctica dominios críticos de la disciplina, exigiendo el dominio en la construcción de interfaces *Full-Stack* reactivas, el modelado transaccional en bases de datos relacionales (PostgreSQL), la segregación de lógicas de negocio, y la automatización del despliegue continuo bajo los estándares y métricas de calidad de la cultura DevOps.

2. Objetivos

2.1 Objetivo General

Diseñar y desarrollar la arquitectura orientada a microservicios para la plataforma Hipstagram, integrando la gestión concurrente de perfiles, contenido multimedia y métricas de interacción social. Esta solución se fundamentará en atributos de calidad de ingeniería de software, garantizando alta disponibilidad, seguridad transaccional y escalabilidad mediante la contenerización con Docker, la automatización del ciclo de vida con un *pipeline* de CI/CD y el aprovisionamiento en la infraestructura de AWS bajo un modelo relacional en PostgreSQL.

2.2 Objetivos Específicos

- Diseñar un perímetro de seguridad perimetral e interno implementando un sistema de autenticación sin estado (stateless) basado en JSON Web Tokens (JWT) y algoritmos criptográficos para el resguardo de credenciales.
- Establecer un esquema de Control de Acceso Basado en Roles (RBAC) que garantice la segregación estricta de privilegios y restrinja la ejecución de los endpoints según los perfiles operativos de usuario final (USER) y administrador (ADMIN).
- Desarrollar el ciclo de vida del contenido multimedia mediante microservicios que orquesten la publicación, almacenamiento y eliminación lógica o física de fotografías, asegurando la consistencia de los metadatos asociados.
- Construir el ecosistema de interacción asíncrona habilitando la emisión de valoraciones binarias (votos like/dislike) con control de idempotencia y concurrencia, junto con un sistema de comentarios paginados.
- Implementar un motor de descubrimiento de contenido optimizado para resolver consultas eficientes basadas en criterios de texto libre y categorización indexada mediante etiquetas (hashtags).
- integrar mecanismos de trazabilidad y gobernanza automatizando la moderación de contenido y construyendo un registro de auditoría (audit log) transaccional, accesible exclusivamente por el rol administrador para el monitoreo de eventos críticos.
- Orquestar el despliegue productivo y la entrega continua mediante la contenerización de los servicios con Docker y Docker Compose, automatizando la integración en un entorno de Amazon Web Services (AWS) para validar la operatividad en la nube.

3. Descripción de Procesos

3.1 Proceso de Registro e Inicio de Sesión

El usuario accede a la aplicación web (React) e ingresa sus datos de registro (nombre, email, contraseña). El API Gateway recibe la petición y la reenvía al auth-service, que valida los datos, aplica hash bcrypt a la contraseña y almacena el usuario en PostgreSQL con rol USER por defecto. En el inicio de sesión, el auth-service valida las credenciales, genera un JWT de 15 minutos y un refresh token de 7 días, retornando ambos al cliente.

3.2 Proceso de Publicación de Fotografía

El usuario selecciona una imagen, escribe una descripción (máximo 128 caracteres) y define hashtags. La petición multipart llega al API Gateway con el JWT del usuario. Se reenvía al post-service, que valida los datos. El media-service sube la imagen a S3 y la somete a moderación (AWS Rekognition). El microservicio de hashtag filter verifica que ningún hashtag esté en la lista de palabras prohibidas. Si todo pasa, se ejecuta una transacción en PostgreSQL: INSERT en posts, INSERT en post_hashtags (con SAVEPOINT) e INSERT en audit_logs, finalizando con COMMIT.

3.3 Proceso de Votación (Like/Dislike)

Un usuario autenticado puede dar like o dislike a publicaciones de otros usuarios. El vote-service valida que no exista ya una votación vigente del mismo usuario para esa publicación (restricción UNIQUE en BD). Si el usuario cambia su reacción, se hace un UPDATE del registro existente y se ajustan los contadores likes_count y dislikes_count en la tabla posts dentro de una transacción atómica.

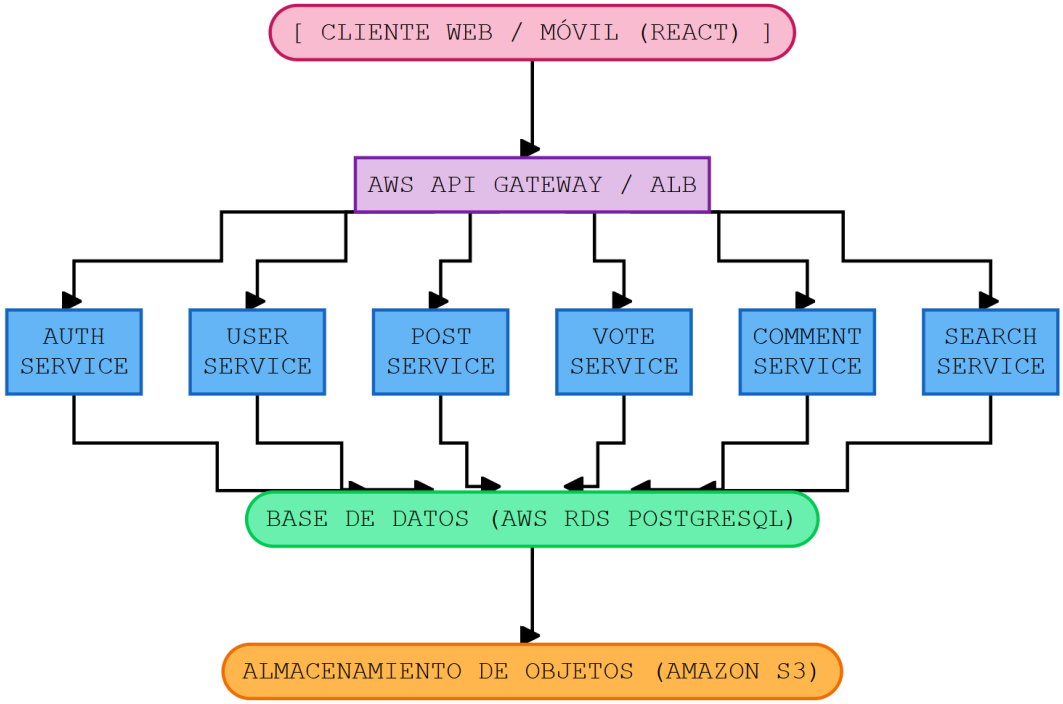
3.4 Proceso de Búsqueda

El search-service ofrece dos modalidades: búsqueda por hashtag (consulta la tabla post_hashtags filtrando por el nombre del hashtag) y búsqueda por texto libre (búsqueda full-text sobre el campo description de posts). Ambos resultados se paginan y se ordenan por likes descendente. El modo Explorar devuelve las publicaciones con más likes globalmente.

3.5 Proceso de Moderación Administrativa

El rol ADMIN accede al panel de administración donde puede ver publicaciones en estado PENDIENTE o BLOQUEADO, aprobarlas o rechazarlas, gestionar usuarios (activar/desactivar), y editar la lista de palabras/hashtags prohibidos en formato JSON. Cada acción queda registrada en audit_logs con el actor, la acción, la entidad afectada y el resultado.

4. Diagramas de bloque de procesos



La infraestructura del sistema Hipstagram está estructurada bajo un modelo de arquitectura distribuida en la nube, optimizada para garantizar escalabilidad, alta disponibilidad y un flujo de datos unidireccional. La topología se divide en las siguientes capas lógicas y tecnológicas:

- **Capa de Presentación (Clientes de Consumo):** Constituida por las interfaces de usuario web y móviles desarrolladas con la biblioteca React. Estos clientes actúan como los emisores de las peticiones HTTP y son el único punto de interacción directa con el usuario final.

- **Capa de Enrutamiento Perimetral:** Implementada a través de un *API Gateway* y/o un Balanceador de Carga de Aplicaciones (ALB) en AWS. Este componente funge como el único punto de contacto (*Single Point of Contact*) hacia el exterior. Su responsabilidad es interceptar el tráfico entrante de los clientes, resolver las reglas de enrutamiento y delegar las peticiones hacia el microservicio correspondiente en el clúster interno.

- **Ecosistema de Microservicios (Capa de Aplicación):** La lógica de negocio está altamente desacoplada en seis servicios independientes, cada uno encapsulando un dominio de negocio específico:
    - **Auth Service:** Gestión de identidades y emisión de tokens (JWT).
    - **User Service:** Administración de perfiles y estados de cuenta.
    - **Post Service:** Orquestación del ciclo de vida de las publicaciones.
    - **Vote Service:** Control de métricas de interacción (likes/dislikes) e idempotencia.
    - **Comment Service:** Gestión de los hilos de respuesta.
    - **Search Service:** Motores de consulta indexada y texto libre.
- **Capa de Persistencia Relacional:** Todos los microservicios convergen hacia una base de datos centralizada alojada en Amazon RDS utilizando el motor PostgreSQL. Este componente garantiza la integridad referencial de los datos estructurales (usuarios, metadatos de posts, conteo de votos) y soporta las transacciones ACID del sistema.
- **Almacenamiento de Objetos Binarios:** Separado de la persistencia relacional, el sistema delega el almacenamiento físico de los activos multimedia (fotografías) a los *buckets* de Amazon S3. Esta abstracción permite que la base de datos relacional se mantenga ligera (almacenando únicamente las URLs o referencias), mientras S3 maneja la entrega eficiente del contenido estático a gran escala.

5. Bloques

Cliente Web/Móvil

- React (Web)
- React Native / PWA
- Envía peticiones HTTP

API Gateway

- Punto único de entrada
- Valida JWT
- Aplica RBAC
- Enruta a microservicios

auth-service

- Registro
- Login
- Refresh token
- Hash contraseña (bcrypt)

user-service

- Gestión de perfil
- Activar/desactivar usuarios

post-service

- Crear publicación
- Eliminar publicación
- Obtener feed

vote-service

- Like / Dislike
- Control UNIQUE(user_id + post_id)

comment-service

- Crear comentario
- Listar comentarios paginados

search-service

- Buscar por hashtag
- Buscar por texto libre
- Explorar (Top likes)

media-service

- Sube imagen a:
    - AWS S3

moderation-service

- Filtra hashtags prohibidos
- Analiza imagen
- Cambia estado del post

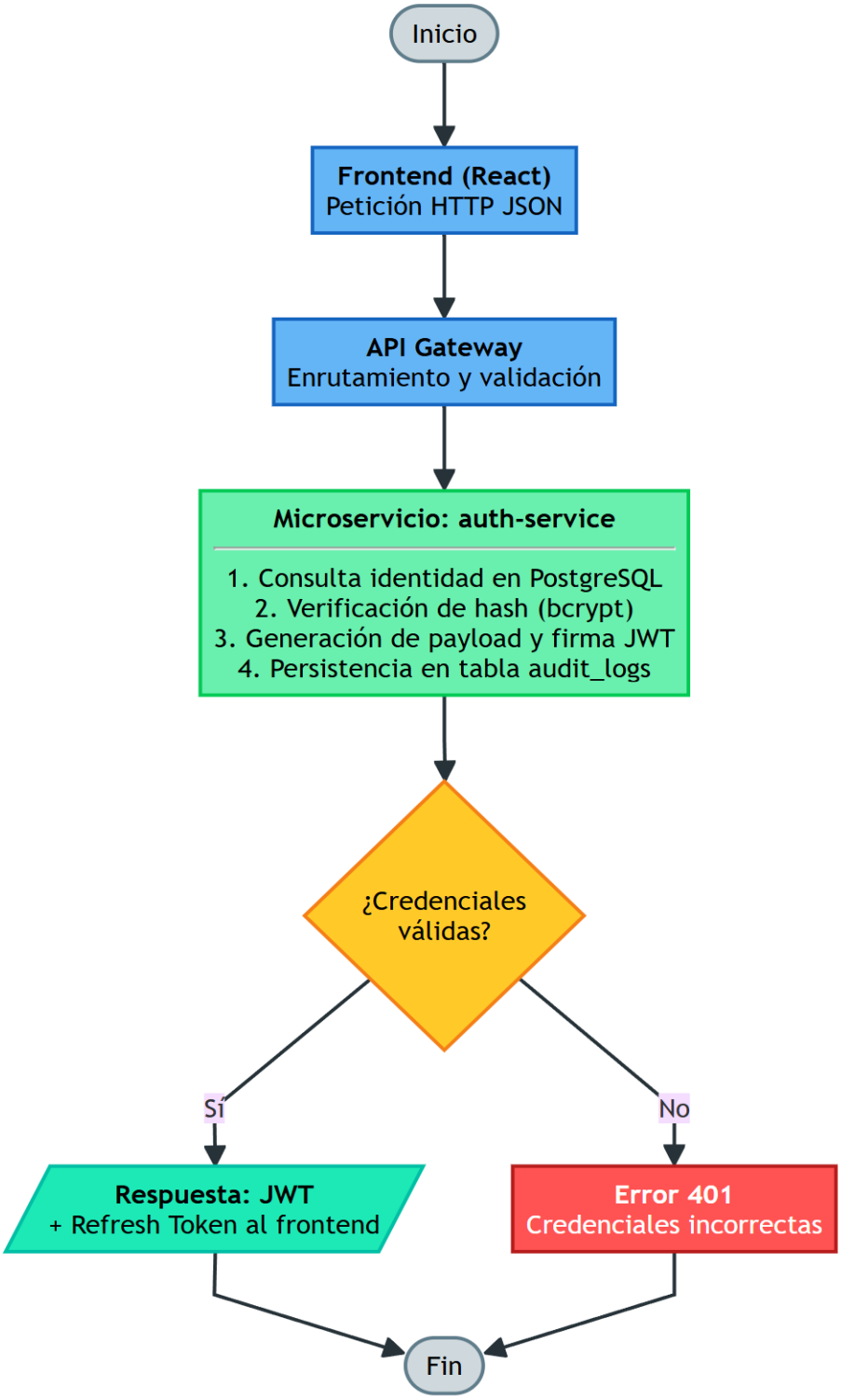
audit-service

- Guarda logs en tabla audit_logs
- Visible solo ADMIN

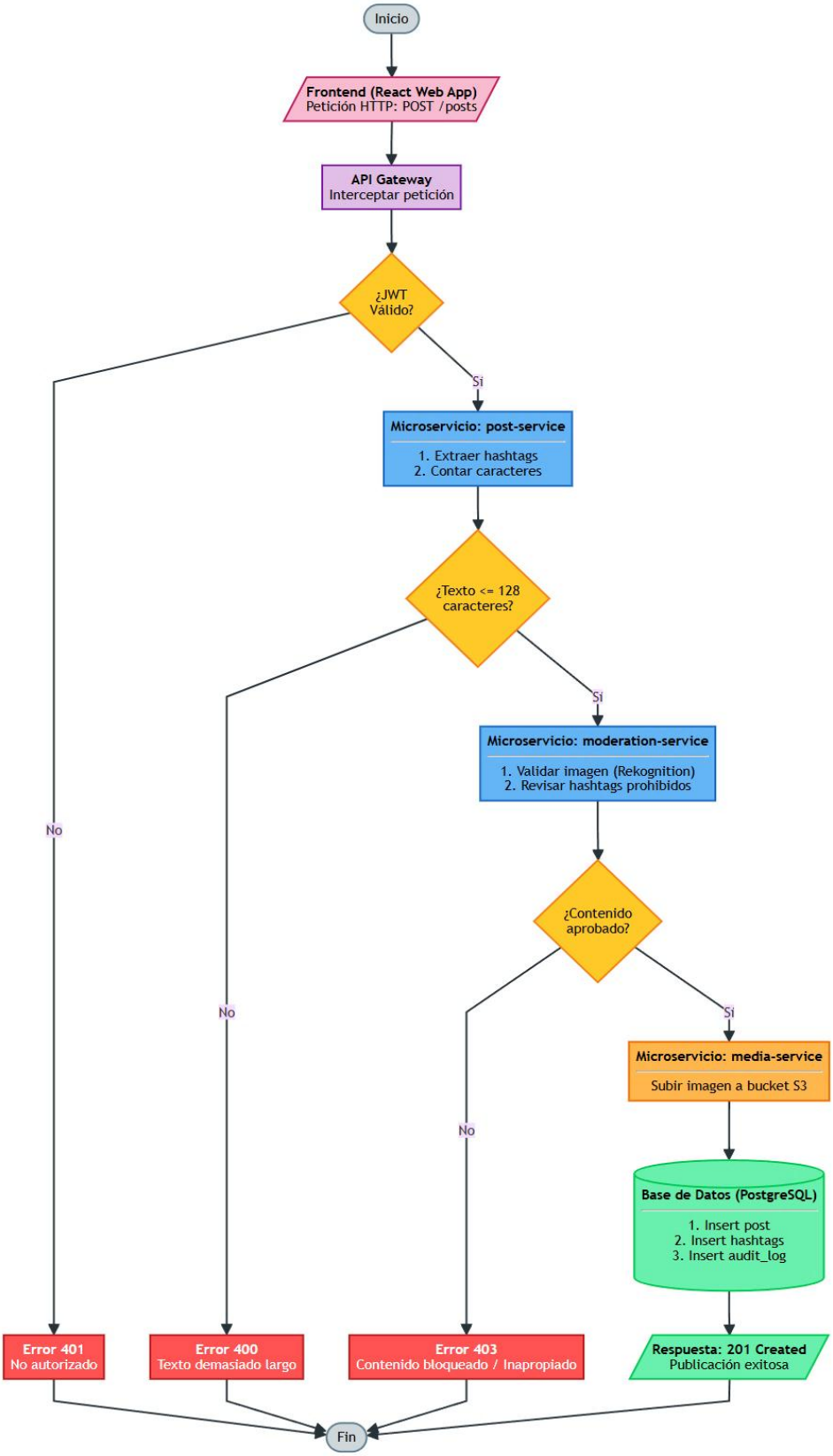
Microservicio	Responsabilidades del bloque
api-gateway	Único punto de entrada. Verifica firma JWT, extrae rol, aplica RBAC por ruta, genera X-Request-ID y reenvía al microservicio interno correspondiente.
auth-service	Registro de usuario (validación + bcrypt). Login con generación de JWT + refresh token. Renovación y revocación de tokens. Auditoría de accesos.
user-service	Consulta y actualización de perfil. Panel ADMIN: activar/desactivar cuentas, gestionar usuarios. Endpoint de audit_logs (solo ADMIN).
post-service	Crear publicación con transacción COMMIT/SAVEPOINT. Eliminar publicación (solo autor). Feed paginado ordenado por likes. Detalle de publicación.
media-service	Recibir imagen multipart. Subir a AWS S3. Llamar AWS Rekognition para moderación automática. Retornar URL al post-service.
vote-service	Registrar like o dislike. Prevenir duplicados con UNIQUE(user_id, post_id). Actualizar contadores de forma atómica. Cambiar reacción existente.
comment-service	Crear comentario con validación y sanitización. Listar comentarios paginados por publicación. Eliminar comentario (autor o ADMIN).
search-service	Búsqueda por hashtag (JOIN). Búsqueda full-text (tsvector). Modo Explorar (Top Likes global). Todos los resultados paginados.

5.1 Procesos

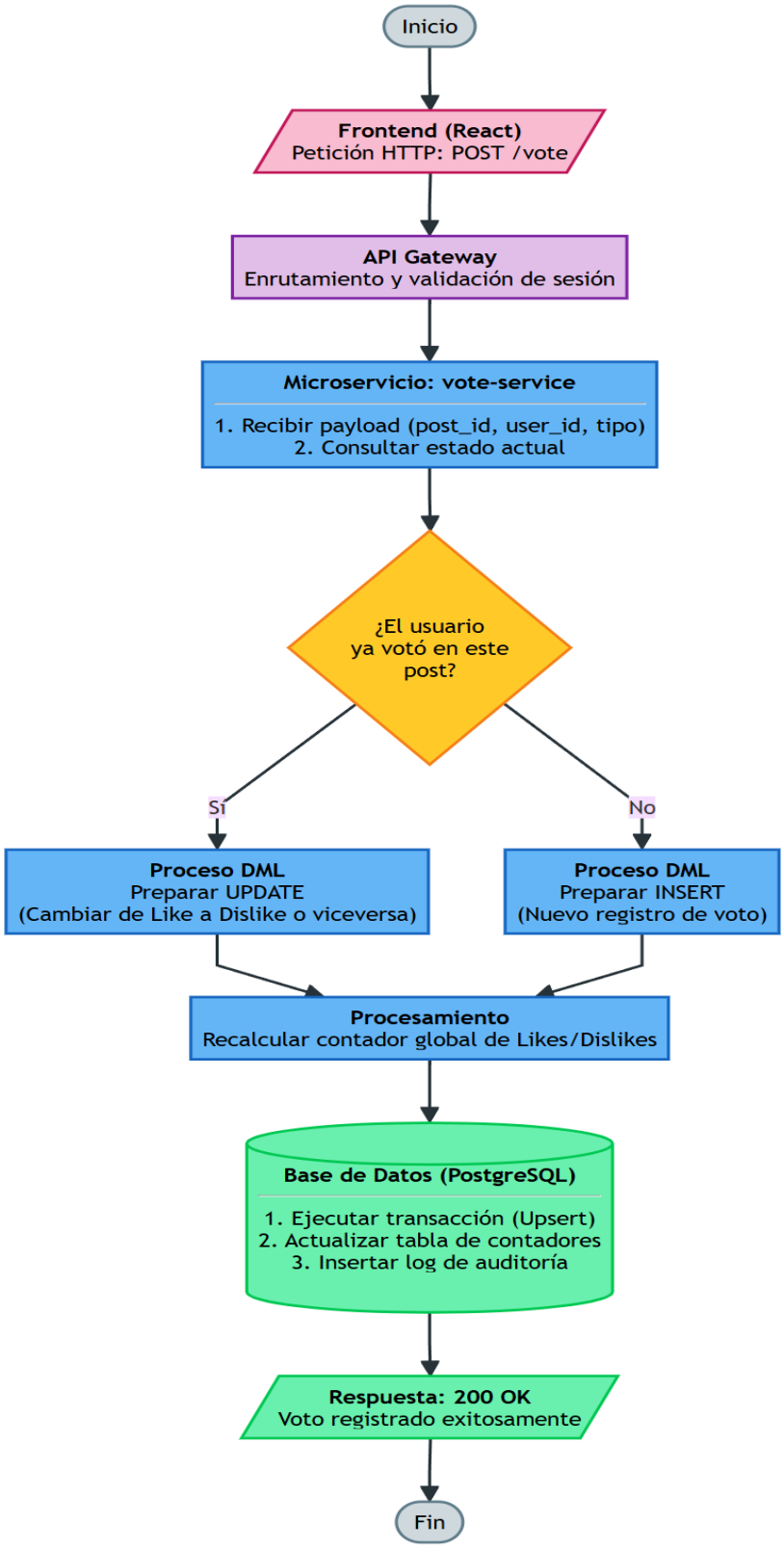
5.1.1 Proceso – Login



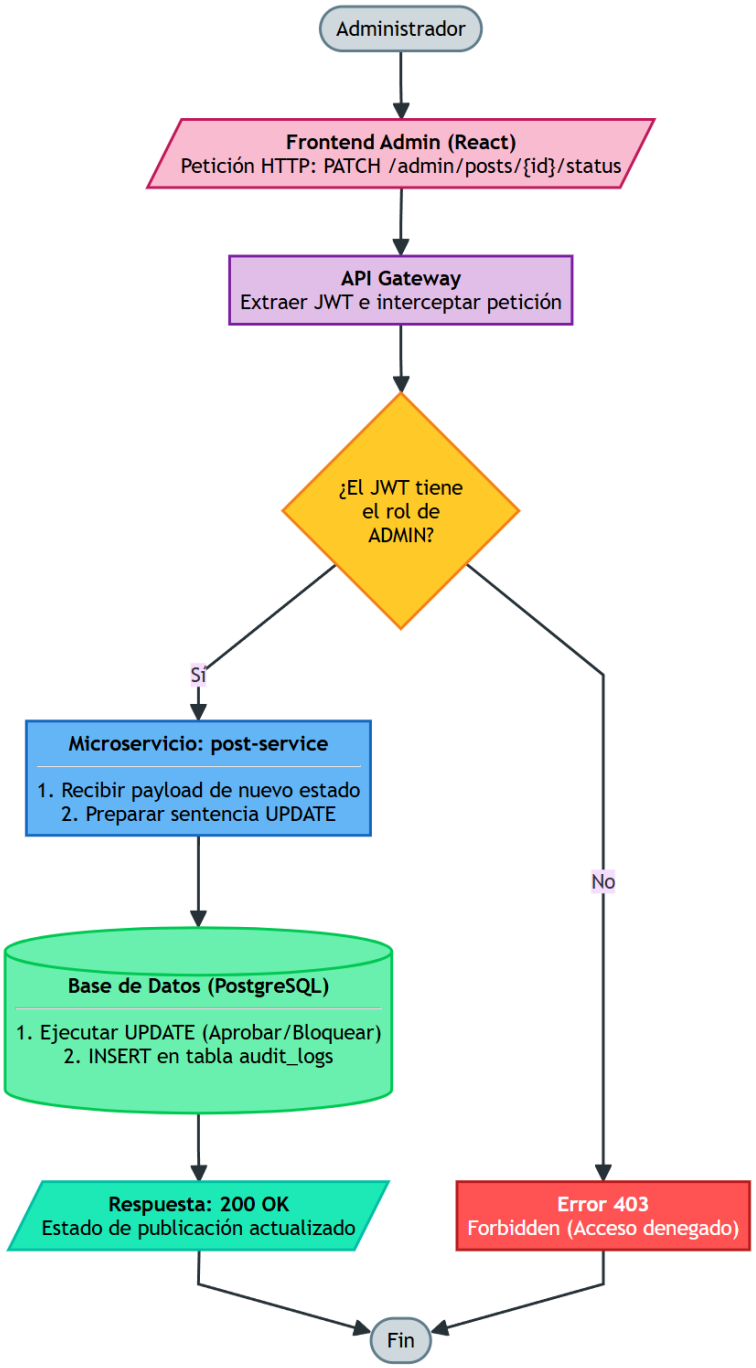
5.1.2 Proceso – Publicar Foto



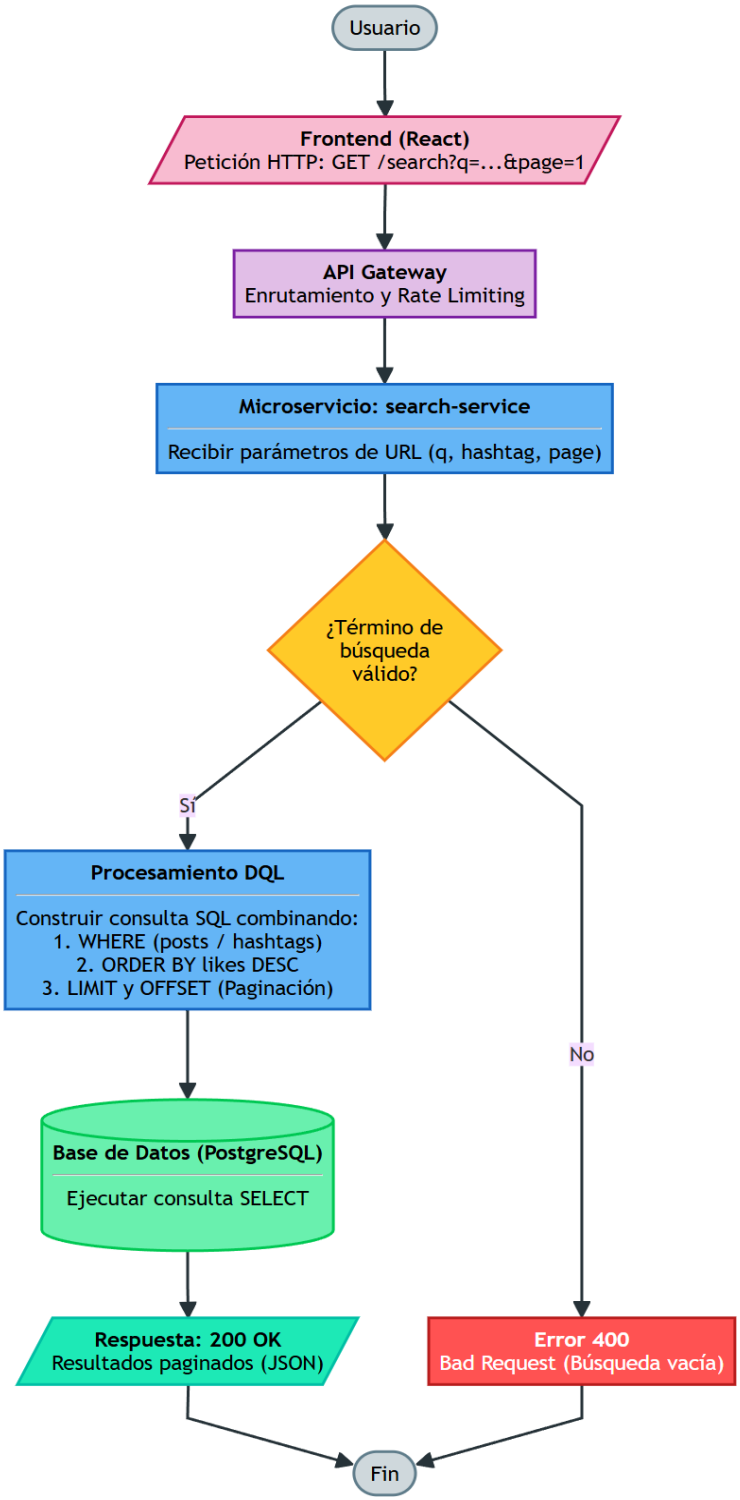
5.1.3. Proceso – Votar (Like/Dislike)



5.1.4. Proceso – Moderación (Admin)



5.1.5. Proceso – Búsqueda



6. Descripción de procedimientos

Esta sección describe paso a paso los procedimientos técnicos de cada proceso principal del sistema. A diferencia de la sección anterior —que describe qué hace cada proceso—, los procedimientos detallan cómo se ejecuta cada paso: las acciones específicas, las validaciones aplicadas, las reglas de negocio y el manejo de errores en cada etapa.

6.1 Registro de Usuario

N°	Actor	Acción y regla de negocio
1	Usuario / React	Completa el formulario (nombre, email, contraseña). React valida que los campos no estén vacíos antes de enviar. Ejecuta POST /auth/register.
2	API Gateway	Recibe la petición en puerto 443, genera X-Request-ID (UUID v4), reenvía al auth-service. Ruta pública: NO requiere JWT.
3	auth-service	Valida body con Joi: name (min 2 chars), email (formato RFC válido), password (min 8 chars). Si falla → HTTP 400 con detalle de errores.
4	auth-service → BD	SELECT WHERE email = $1. Si ya existe → HTTP 409 Conflict: "El correo ya está registrado". Si no, ejecuta bcrypt.hash(password, 12).
5	auth-service → BD	INSERT INTO users (name, email, password_hash, role='USER'). INSERT INTO audit_logs (action='USER_REGISTERED', result='SUCCESS'). COMMIT.
6	auth-service → React	HTTP 201 Created. Retorna objeto de usuario (sin password_hash). React redirige a la pantalla de Login.

6.2 Inicio de Sesión

N°	Actor	Acción y regla de negocio
1	Usuario / React	Ingresa email y contraseña. React ejecuta POST /auth/login.
2	auth-service	SELECT * FROM users WHERE email=$1. Si no existe o is_active=FALSE → audit_logs LOGIN_FAILED → HTTP 401 (mensaje genérico para no revelar existencia).
3	auth-service	bcrypt.compare(password, hash). Si no coincide → audit_logs LOGIN_FAILED → HTTP 401 (mismo mensaje genérico).
4	auth-service	jwt.sign({ userId, role }, JWT_SECRET, { expiresIn: '15m' }). Genera refreshToken (UUID + JWT firmado) con expiración 7 días.
5	auth-service → BD	INSERT INTO refresh_tokens (user_id, token, expires_at). INSERT INTO audit_logs (action='LOGIN_SUCCESS'). COMMIT.
6	auth-service → React	HTTP 200 con { accessToken, refreshToken, user: { id, name, role } }. React almacena tokens y redirige al Feed.

6.3 Publicación de Fotografía

N°	Actor	Acción y regla de negocio
1	Usuario / React	Selecciona imagen, escribe descripción (máx. 128 chars) y hashtags. React construye FormData. POST /posts con Authorization: Bearer <token>.
2	API Gateway	jwt.verify(token, JWT_SECRET). Extrae userId, role='USER'. Genera X-Request-ID. Añade headers x-user-id, x-user-role. Reenvía al post-service.
3	post-service	Valida body: description máx. 128 chars, hashtags como array de strings. Si falla → HTTP 400.

4	post-service → media-service	Envía imagen al media-service. Este ejecuta PutObjectCommand a S3 y llama Rekognition.DetectModerationLabels. Si hay contenido inapropiado → HTTP 422: "Imagen rechazada".
5	post-service → BD	SELECT word FROM banned_words WHERE word = ANY($hashtags). Si algún hashtag está prohibido → HTTP 422: "Hashtag prohibido: #xxx".
6	post-service → BD	BEGIN; INSERT INTO posts (..., status='PUBLISHED'); SAVEPOINT sp_ht; INSERT INTO post_hashtags; INSERT INTO audit_logs; COMMIT. Si falla hashtags → ROLLBACK TO SAVEPOINT sp_ht.
7	post-service → React	HTTP 201 con objeto del post creado (incluye URL de S3). React agrega la publicación al feed.

6.4 Votación (Like/Dislike)

N°	Actor	Acción y regla de negocio
1	Usuario / React	Presiona Like o Dislike. POST /votes con { postId, type } y JWT.
2	API Gateway	Verifica JWT. Solo roles USER y ADMIN pueden votar. Reenvía al vote-service.
3	vote-service	Verifica que el userId del JWT no sea el dueño del post → HTTP 403: "No puedes votar tu propia publicación".
4	vote-service → BD	INSERT INTO votes ... ON CONFLICT (user_id, post_id) DO UPDATE SET type=EXCLUDED.type. La restricción UNIQUE garantiza máximo 1 reacción activa.
5	vote-service → BD	UPDATE posts SET likes_count=(subq COUNT LIKE), dislikes_count=(subq COUNT DISLIKE). INSERT audit_logs. COMMIT.
6	vote-service → React	HTTP 200 con { likesCount, dislikesCount }. React actualiza contadores en UI sin recargar.

6.5 Creación de Comentario

N°	Actor	Acción y regla de negocio
1	Usuario / React	Escribe comentario y presiona Enviar. POST /comments con { postId, content } y JWT.
2	comment-service	Valida content: entre 1 y 500 caracteres. Aplica sanitización (strip HTML/scripts). Si falla → HTTP 400.
3	comment-service → BD	Verifica que el post exista y sea status='PUBLISHED'. INSERT INTO comments. INSERT audit_logs. COMMIT. → HTTP 201.

6.6 Búsqueda de Contenido

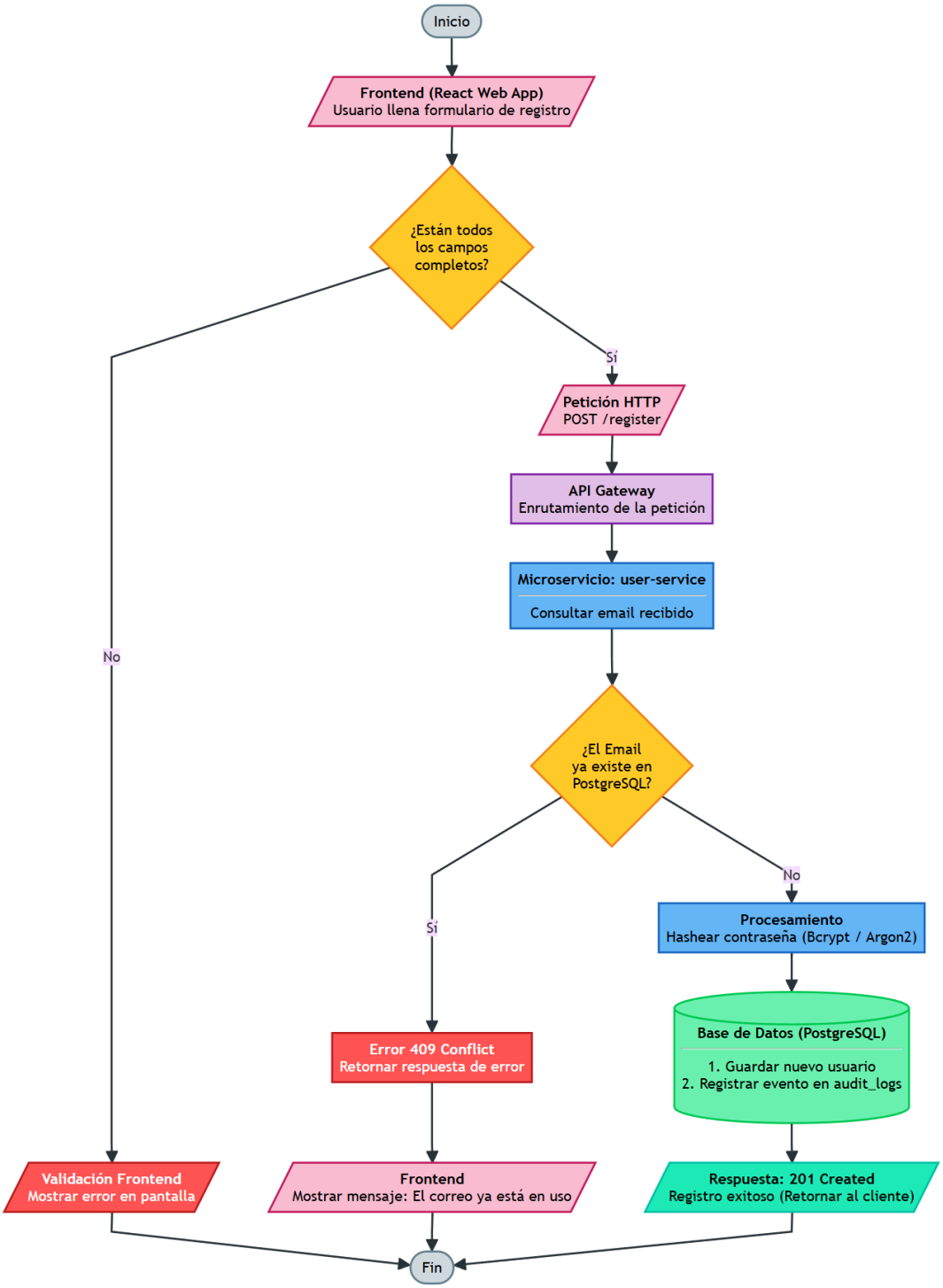
N°	Actor	Acción y regla de negocio
1	Usuario / React	Ingresa término. React detecta si inicia con '#' para determinar tipo. GET /search?q=...&type=hashtag|text y parámetros de paginación.
2a	search-service (hashtag)	JOIN posts + post_hashtags + hashtags WHERE h.name=$1 AND status='PUBLISHED'. Ordena por likes_count DESC. Pagina con LIMIT/OFFSET.
2b	search-service (texto libre)	WHERE to_tsvector('spanish', description) @@ plainto_tsquery('spanish', $1). Ordena por ts_rank DESC, luego likes DESC. Pagina.
3	search-service → React	HTTP 200 con { results, total, page, limit }. React renderiza tarjetas paginadas. Si results=[] → mensaje "Sin resultados".

6.7 Moderación Administrativa

N°	Actor	Acción y regla de negocio
1	ADMIN / React	Accede al panel de moderación. GET /admin/posts?status=PENDING con JWT del ADMIN.
2	API Gateway	Verifica JWT. Si role≠'ADMIN' → HTTP 403 Forbidden. Si ADMIN → reenvía al user-service.
3	ADMIN elige acción	Selecciona: PUBLISHED (aprobar), REJECTED (rechazar) o BLOCKED (bloquear). Puede añadir razón.
4	user-service → BD	BEGIN; UPDATE posts SET status=$1 WHERE id=$2; INSERT audit_logs (actor_role='ADMIN', action='POST_BLOCKED|APPROVED', result='SUCCESS'); COMMIT.
5	user-service → React	HTTP 200. Panel admin actualiza la lista y retira la publicación de la cola de revisión.

7. Diagramas de flujos del proceso

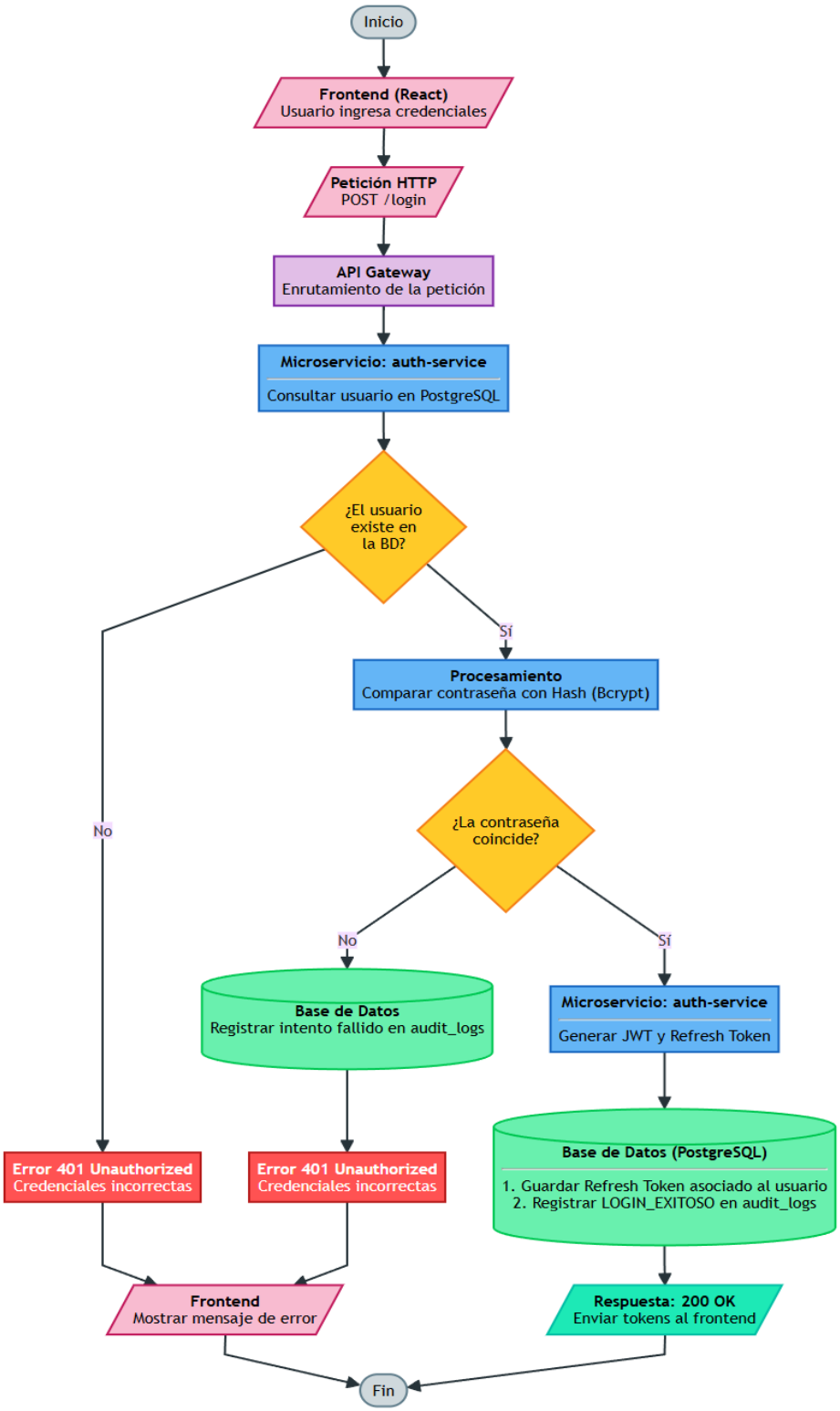
7.1 Diagrama De Flujo – Registro De Usuario



El flujo transaccional para el aprovisionamiento de nuevas identidades se inicializa en la capa de presentación (aplicación web en React), donde se capturan las credenciales del usuario. En esta etapa perimetral, se ejecuta una validación estática para asegurar la completitud de los datos de entrada. Una vez superada esta prevalidación, el cliente emite una petición HTTP POST hacia el API Gateway, componente que asume la responsabilidad de enrutar la solicitud hacia el microservicio especializado (user-service).

Dentro de la capa de aplicación, el microservicio interroga al motor relacional (PostgreSQL) para verificar la disponibilidad del correo electrónico y garantizar la unicidad de la entidad. Si se detecta una colisión de registros, el flujo se interrumpe y el servidor retorna un código de estado HTTP 409 Conflict, instruyendo al frontend a desplegar la notificación de redundancia correspondiente. Por el contrario, si la validación de disponibilidad es exitosa, se ejecuta el procesamiento criptográfico de la contraseña aplicando algoritmos de hashing seguro (Bcrypt o Argon2). Finalmente, la transacción culmina con la persistencia del nuevo usuario en la base de datos y la inserción concurrente del evento en la bitácora del sistema (audit_logs), garantizando así la trazabilidad de la operación. El ciclo de vida de la petición se cierra retornando un código HTTP 201 Created al cliente, confirmando la creación íntegra de la cuenta.

7.2. Diagrama De Flujo – Login

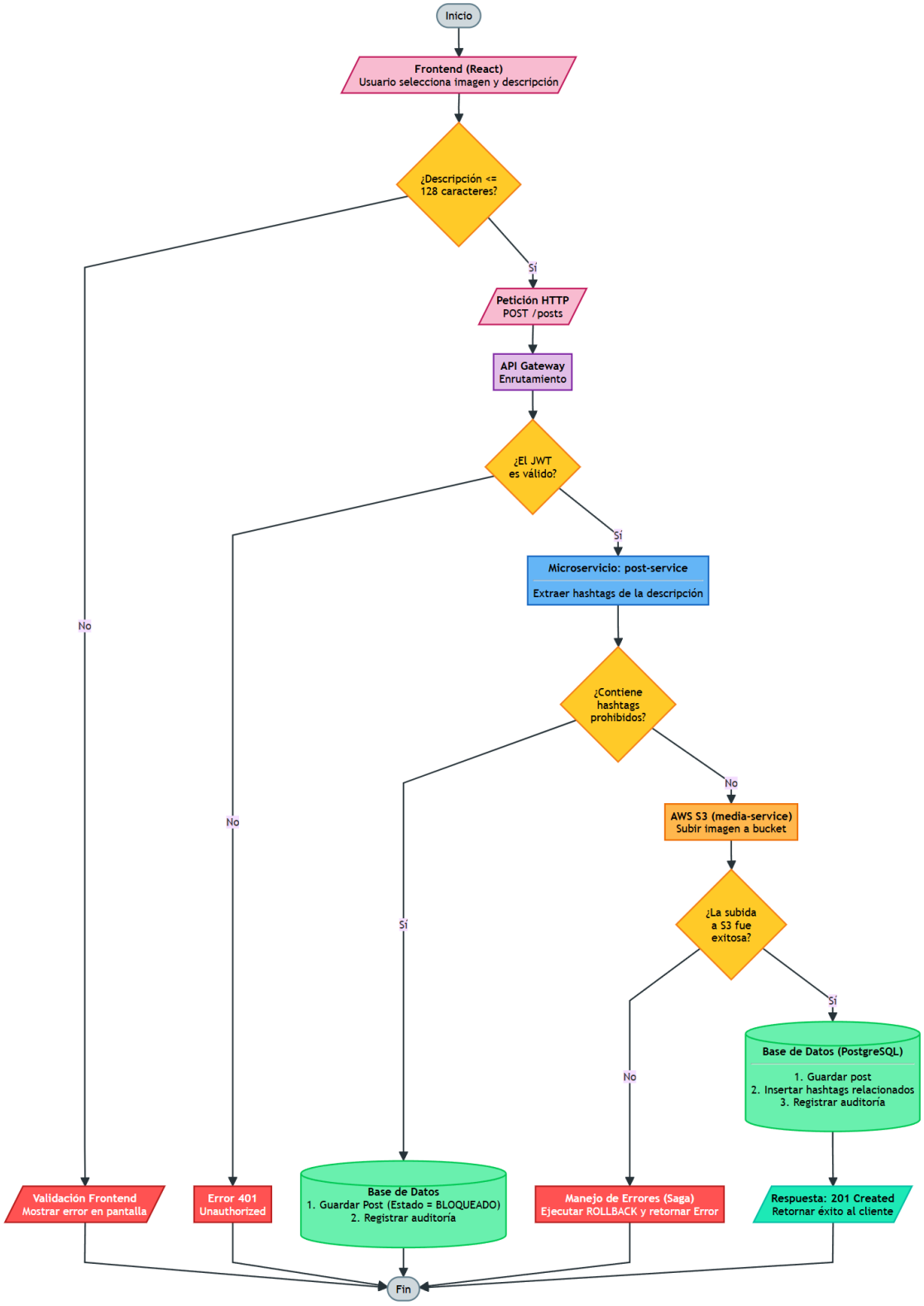


El flujo transaccional de autenticación se inicializa en la capa de presentación (aplicación en React), donde se capturan las credenciales de acceso para emitir una petición HTTP POST dirigida al endpoint /login. Esta solicitud perimetral es interceptada por el API Gateway, el cual asume la responsabilidad de enrutar el tráfico hacia el microservicio de identidad (auth-service). En la capa de aplicación, este servicio consulta el motor relacional (PostgreSQL) para verificar la existencia del registro del usuario. Si la entidad no es localizada, el flujo se interrumpe de inmediato emitiendo un código de estado HTTP 401 Unauthorized.

En caso de que el usuario exista, se procede a la validación criptográfica de la credencial, comparando la contraseña en texto plano contra el hash almacenado utilizando el algoritmo Bcrypt. Una resolución negativa en esta comprobación desencadena la inserción de un evento de intento fallido en la bitácora del sistema (audit_logs) para fines de trazabilidad de seguridad, previo a denegar el acceso retornando el error 401 al cliente.

Por el contrario, si la validación criptográfica es exitosa, el microservicio implementa la autorización sin estado (stateless) mediante la emisión de un JSON Web Token (JWT) y un Refresh Token. La operación culmina garantizando la persistencia del Refresh Token vinculado al usuario en la base de datos, junto con el registro del evento LOGIN_EXITOSO en el módulo de auditoría. El ciclo de vida de la petición se cierra retornando un código HTTP 200 OK, despachando los tokens hacia el frontend para autorizar la navegación y la ejecución de funcionalidades acordes al perfil de la cuenta.

7.3 Diagrama De Flujo – Publicar Foto

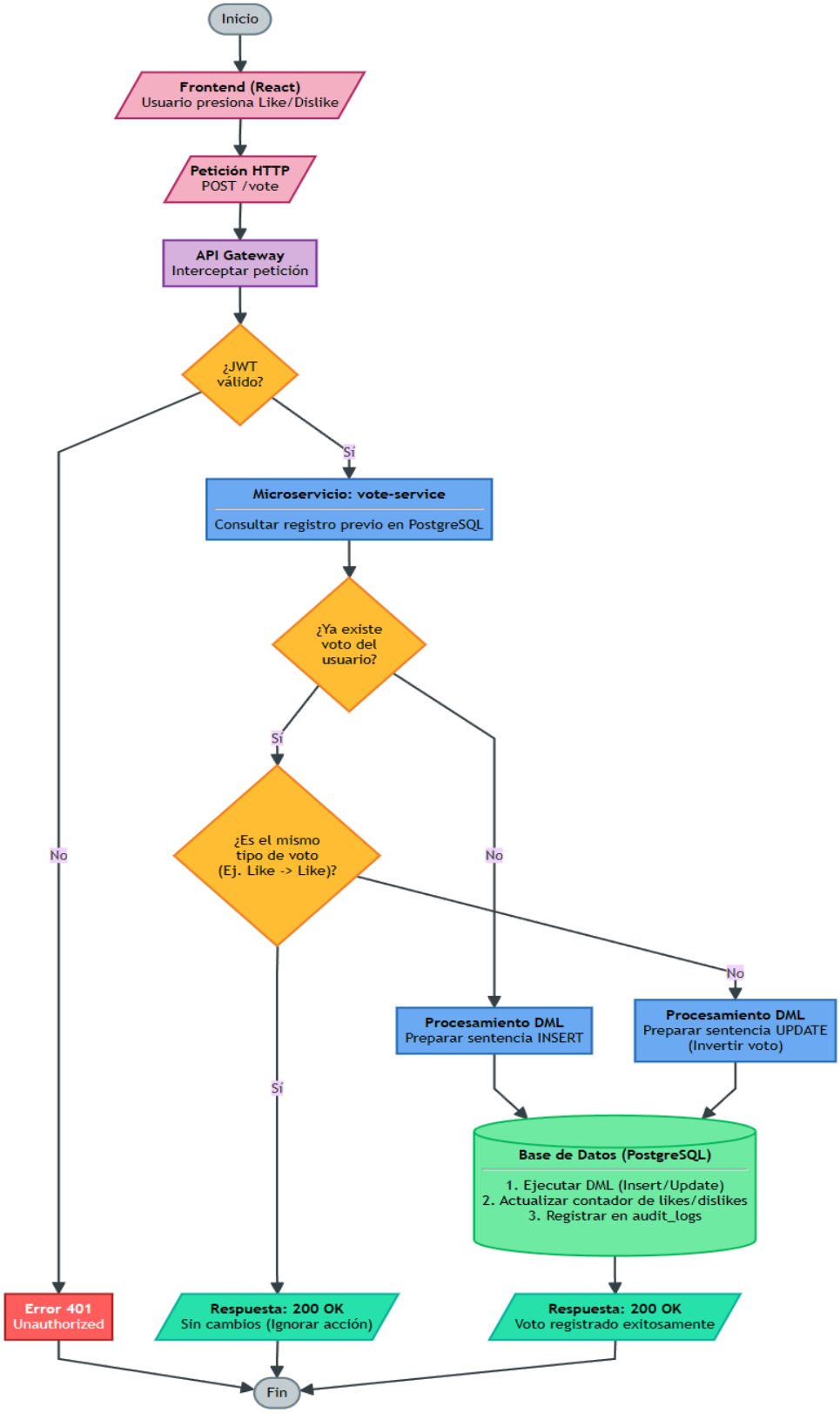


El flujo transaccional para la creación de contenido multimedia se inicializa en la capa de presentación (React), donde se ejecuta una prevalidación estática para garantizar que la longitud de la descripción no exceda el límite de 128 caracteres. Superada esta restricción perimetral, el cliente emite una petición HTTP POST /posts que es interceptada por el *API Gateway*. Este componente valida la firma e integridad del JSON Web Token (JWT); si la verificación falla, el ciclo se interrumpe inmediatamente retornando un código de estado HTTP *401 Unauthorized*.

Una vez autorizada la solicitud, la petición es enrutada al microservicio encargado (post-service), el cual extrae los *hashtags* de la carga útil (*payload*) y ejecuta una validación de reglas de negocio contra un diccionario de términos prohibidos. Si el motor de moderación detecta contenido indebido, el sistema persiste la publicación en la base de datos PostgreSQL bajo un estado restringido (BLOQUEADO), genera el registro en el log de auditoría y finaliza la operación.

Si la validación semántica es exitosa, el flujo delega la transferencia del objeto binario (fotografía) al microservicio de almacenamiento (media-service), el cual gestiona la subida hacia un *bucket* en Amazon S3. Para asegurar la consistencia de los datos en esta arquitectura distribuida, el sistema implementa un mecanismo de compensación de errores (patrón Saga); si la comunicación con AWS fracasa, se ejecuta un ROLLBACK y se retorna una excepción controlada. Por el contrario, si la carga en la nube es confirmada, se consolida una transacción relacional en PostgreSQL que almacena la publicación, vincula los *hashtags* correspondientes e inserta el evento en la bitácora de auditoría (audit_logs). El proceso culmina exitosamente despachando un código HTTP *201 Created* hacia el *frontend*.

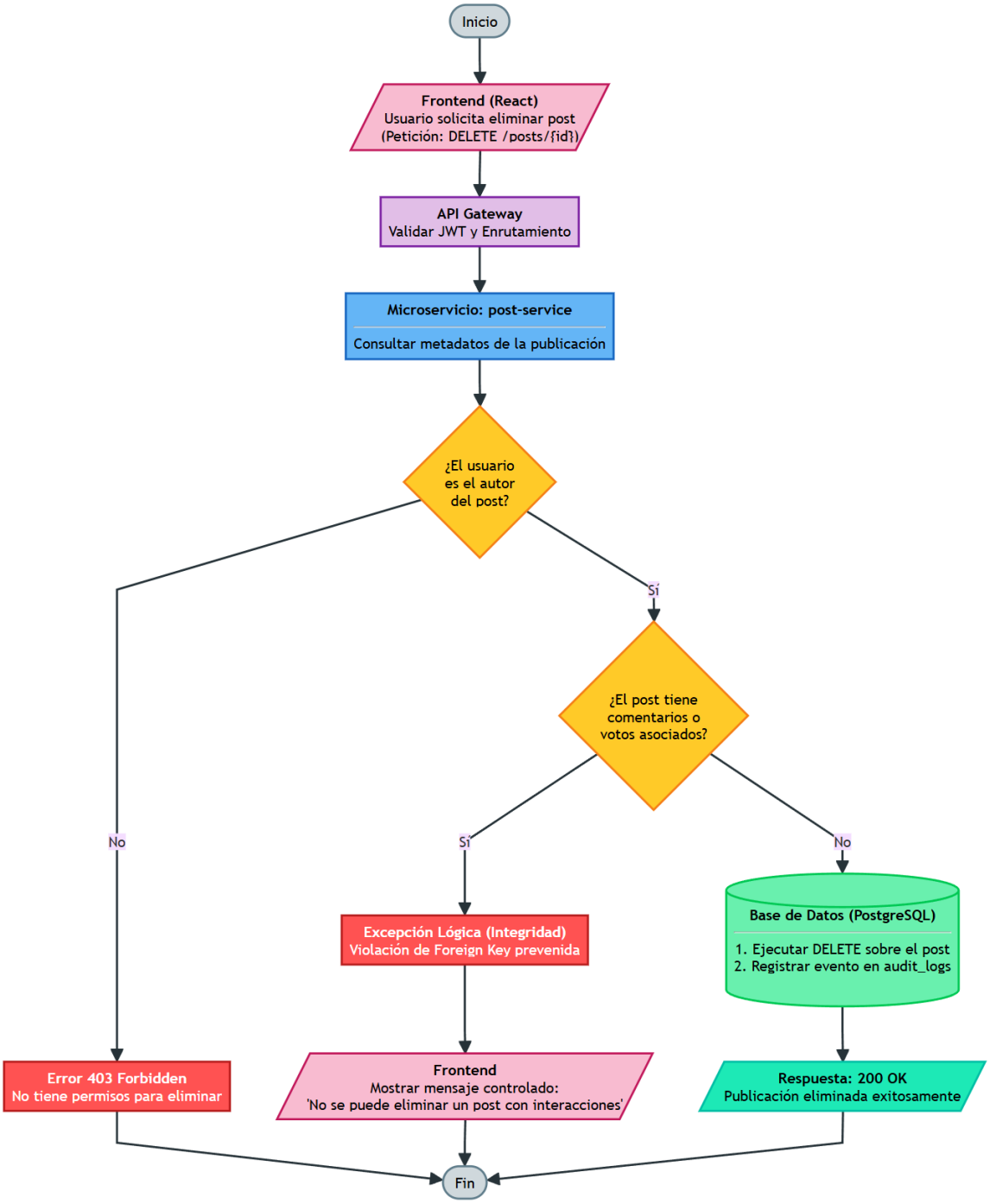
7.4 Diagrama De Flujo – Votar (Like / Dislike)



El flujo transaccional para la emisión de métricas de interacción (votos like o dislike) se inicializa en la capa de presentación (React) mediante la emisión de una petición HTTP POST /vote. Esta solicitud es interceptada por el API Gateway, encargado de validar la integridad y firma del JSON Web Token (JWT); si la credencial carece de validez, el ciclo se interrumpe inmediatamente retornando un estado HTTP 401 Unauthorized.

Una vez autorizada la solicitud, la petición es delegada al microservicio especializado (vote-service), el cual consulta el motor relacional (PostgreSQL) para verificar si el usuario posee un registro previo asociado a esa publicación. Para garantizar la consistencia de los datos y aplicar el control de idempotencia, si el sistema detecta que el usuario intenta emitir exactamente la misma reacción que ya está activa, se ignora la mutación y se retorna un código HTTP 200 OK sin alterar la base de datos. Si no existe un voto previo, el microservicio prepara una sentencia DML de tipo INSERT; alternativamente, si existe una reacción de tipo opuesto (inversión de voto), se formula una instrucción UPDATE. Finalmente, la operación se materializa consolidando la transacción en la base de datos, lo que incluye la ejecución del DML, la actualización de los contadores agregados de la publicación y la inserción del evento en la bitácora (audit_logs), culminando con una respuesta HTTP 200 OK que confirma al cliente el registro exitoso de la interacción.

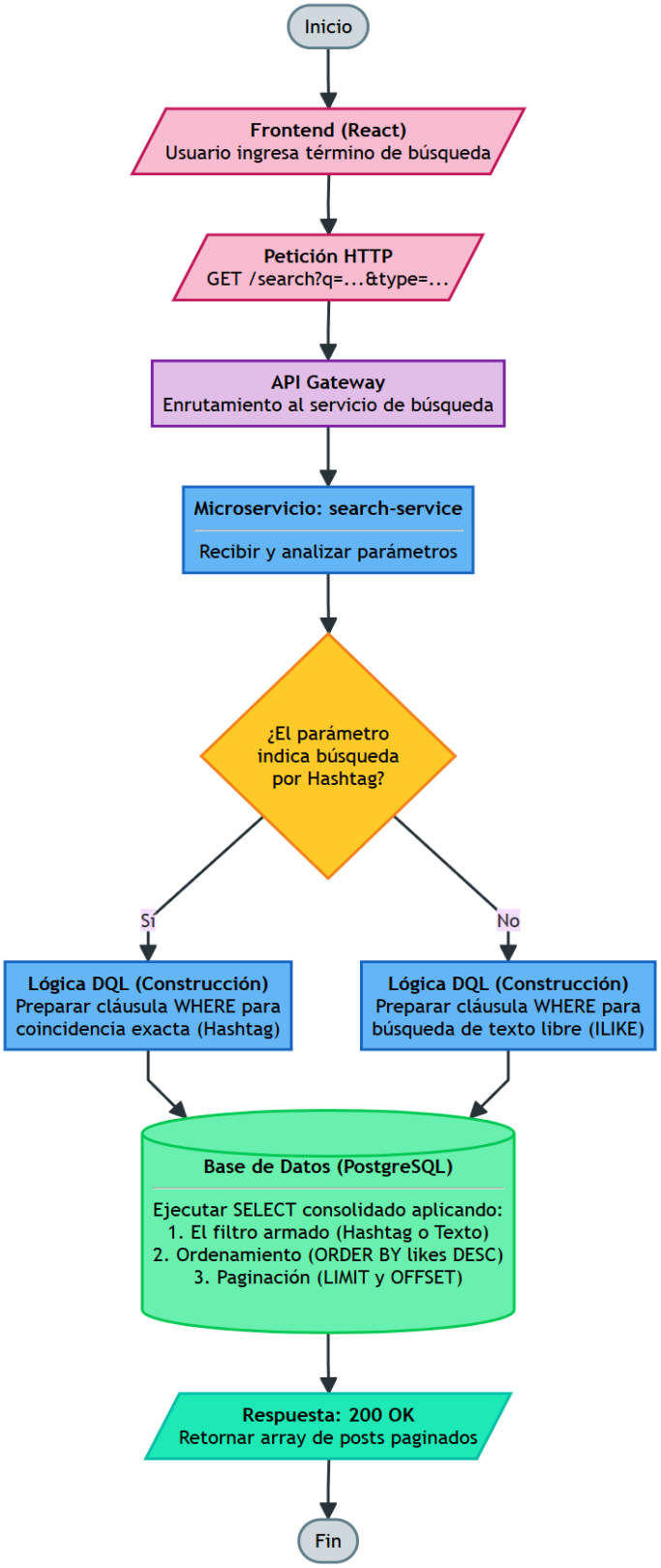
7.5 Diagrama De Flujo – Eliminar Publicación



El flujo transaccional para la eliminación de publicaciones se inicializa en la capa de presentación (React) mediante la emisión de una petición HTTP DELETE /posts/{id}. Esta solicitud es interceptada por el *API Gateway*, el cual ejecuta la validación estructural del JSON Web Token (JWT) y enruta el tráfico hacia el microservicio encargado (post-service). En la capa de aplicación, el servicio consulta los metadatos del registro para evaluar la primera política de control de acceso: verificar la autoría del recurso. Si el identificador del usuario solicitante no coincide con el del autor original, la transacción se aborta, denegando la operación y retornando un código de estado HTTP *403 Forbidden*.

Superada la validación de autoría, el flujo transita hacia la evaluación de la integridad referencial. El sistema verifica si la publicación posee entidades dependientes activas, específicamente métricas de interacción o hilos de comentarios. Para prevenir una violación restrictiva de llaves foráneas (*Foreign Keys*) a nivel de PostgreSQL, si el recurso contiene dependencias, el *backend* captura la excepción lógica y transfiere una respuesta controlada a la interfaz de usuario, indicando la imposibilidad de suprimir un registro con interacciones. Por el contrario, si la publicación carece de asociaciones, el microservicio ejecuta la sentencia DML DELETE directamente en la base de datos y registra de forma síncrona el evento de eliminación en la bitácora del sistema (audit_logs). El ciclo culmina exitosamente despachando un código HTTP *200 OK* hacia el cliente.

7.6 Diagrama De Flujo – Búsqueda



El flujo transaccional del motor de búsqueda se inicializa en la capa de presentación (React), donde se captura el criterio de exploración ingresado por el usuario. Esta acción detona una petición HTTP GET parametrizada (estructurada mediante query strings como ?q=...&type=...), la cual es interceptada por el API Gateway y enrutada hacia el microservicio especializado (search-service). En la capa de aplicación, el servicio analiza los parámetros de la solicitud para determinar la estrategia de recuperación de información.

Si el algoritmo identifica que la petición corresponde a una categorización por etiqueta (hashtag), el sistema construye dinámicamente una sentencia DQL (Data Query Language) implementando una cláusula WHERE para evaluar una coincidencia exacta. Alternativamente, si el parámetro indica una búsqueda por texto libre, la lógica de construcción prepara una consulta difusa optimizada para el motor PostgreSQL, utilizando el operador ILIKE para una evaluación insensible a mayúsculas y minúsculas (case-insensitive).

Finalmente, la operación se traslada a la capa de infraestructura, donde el motor relacional ejecuta la instrucción SELECT consolidada. Para garantizar la eficiencia en el consumo de recursos y soportar altos volúmenes de información, la consulta aplica un ordenamiento jerárquico basado en métricas de popularidad (ORDER BY likes DESC) e implementa una partición de los resultados directamente en la base de datos a través de las cláusulas LIMIT y OFFSET. El ciclo de procesamiento concluye de manera exitosa despachando un código de estado HTTP 200 OK, retornando al cliente una estructura de datos (arreglo) con las publicaciones debidamente paginadas para su renderizado.

8. Listado de requerimientos funcionales de la aplicación.

8.1 Requerimientos Funcionales

1.1. Rol Usuario

1.1.1 Registro de usuario

1.1.2 Inicio y cierre de sesión

1.1.3 Publicar fotografía con descripción (máx. 128 chars) y hashtags

1.1.4 Eliminar fotografía propia

1.1.5 Ver feed ordenado por cantidad de likes

1.1.6 Modo Explorar — ranking global de publicaciones más populares

1.1.7 Dar like o dislike a publicaciones

1.1.8 Comentar publicaciones y ver comentarios

1.1.9 Ver detalle completo de una publicación

1.1.10 Búsqueda de publicaciones por hashtag

1.1.11 Ver perfil propio y perfiles públicos de otros usuarios

1.2. Rol Administrador

1.2.1 Acceder al panel de administración

1.2.2 Gestionar publicaciones (aprobar, rechazar o bloquear)

1.2.3 Gestionar usuarios (activar o desactivar cuentas)

1.2.4 Administrar lista de palabras prohibidas (blacklist)

1.2.5 Consultar log de auditoría del sistema

9. Definición del alcance

El perímetro del proyecto Hipstagram contempla el diseño, desarrollo y aprovisionamiento de una plataforma social distribuida, con interfaces omnicanal (web y móvil), orientada a la gestión y descubrimiento de contenido multimedia. A nivel estructural, la solución se fundamentará en una arquitectura orientada a microservicios, contenerizada íntegramente mediante Docker y orquestada para su despliegue productivo en la infraestructura en la nube de Amazon Web Services (AWS). Esto se complementa con la automatización del ciclo de vida del software a través de un *pipeline* de Integración y Entrega Continua (CI/CD). A nivel operativo, el alcance funcional está delimitado por un modelo de Control de Acceso Basado en Roles (RBAC) que segmenta la operatividad del sistema en dos perfiles estrictamente definidos

9.1 Apartado del alcance para el rol de usuario.

Este perfil delimita las interacciones operativas del usuario final dentro de la plataforma. Sus capacidades incluyen la gestión integral de su ciclo de identidad mediante un flujo de registro y autenticación segura , así como la creación y eliminación lógica de sus propias fotografías. Estas publicaciones podrán estar vinculadas a metadatos descriptivos (restringidos a un límite de 128 caracteres) y etiquetas de indexación (*hashtags*). Adicionalmente, el alcance para este actor contempla la participación en el ecosistema social mediante métricas de interacción asíncrona (*likes/dislikes*) e hilos de comentarios , junto con el acceso a motores de búsqueda (por texto libre o *hashtag*) , la navegación de perfiles públicos y la exploración de un *feed* de contenido jerarquizado por métricas de popularidad.

9.2 Apartado del alcance del ron administrador

Este perfil define el perímetro de administración, dotando a los operadores autorizados de acceso a un panel de control exclusivo e independiente para salvaguardar la integridad de la red social. Las facultades operativas de este rol se centran en la moderación del contenido multimedia (permitiendo bloquear, rechazar o aprobar publicaciones) , así como en la gestión del estado de las cuentas de la comunidad, habilitando la activación o restricción de usuarios. Asimismo, el alcance administrativo incluye la configuración de filtros semánticos mediante la manipulación de diccionarios de restricción (*blacklist* de palabras prohibidas en formato JSON) y la inspección analítica del *log* de auditoría transaccional para auditar la trazabilidad de los eventos críticos generados en el sistema..

10. Identificación de módulos que integran la aplicación.

10.1 Módulo API Gateway

Este módulo actúa como el punto de entrada para clientes web, es decir este módulo se encarga de la validación estricta de inputs y del control de acceso basado en roles.

Es el único componente del sistema expuesto al exterior, operando en el puerto 3000 en desarrollo y puerto 443 en producción. Toda solicitud proveniente del cliente React debe pasar obligatoriamente por este módulo antes de alcanzar cualquier microservicio interno. Entre sus responsabilidades se incluyen la verificación criptográfica de los tokens JWT mediante la función jwt.verify(), la extracción del identificador de usuario y su rol para aplicar el control de acceso por roles (RBAC) en cada ruta, la generación de un identificador único de solicitud (X-Request-ID) para trazabilidad, la aplicación de rate-limiting con un límite de 100 solicitudes por minuto por dirección IP para prevenir abusos, y la configuración de políticas CORS. Ningún microservicio interno es accesible directamente desde el exterior, lo que centraliza la lógica de seguridad y evita su dispersión a lo largo del sistema.

10.2 Módulo de Autenticación de Usuarios

Este módulo gestiona el acceso mediante inicio de sesión seguro, a través de contraseñas, así como la información de los perfiles y el rol que estos ocupan.

Internamente se divide en dos servicios coordinados: el auth-service, responsable del ciclo completo de credenciales, y el user-service, responsable de la gestión de perfiles. El auth-service implementa el registro de nuevos usuarios con validación de datos mediante Joi y hashing de contraseñas con bcrypt a un factor de costo de 12 rondas, garantizando que ninguna contraseña sea almacenada en texto plano. Para la gestión de sesiones implementa un esquema de doble token: un Access Token JWT de corta duración (15 minutos) transmitido en el header de autorización, y un Refresh Token de larga duración (7 días) almacenado en una cookie HttpOnly inaccesible para scripts del navegador. Cada acción relevante — registro, inicio de sesión exitoso, inicio de sesión fallido y cierre de sesión — queda registrada en la tabla de auditoría del sistema. El user-service complementa este módulo gestionando la consulta y actualización de perfiles, así como las operaciones de activación y desactivación de cuentas por parte del administrador.

10.3 Módulo para Publicaciones

Se encarga de permitir la publicación y eliminación de fotografías con descripciones y hashtags, este módulo también gestiona la integración con el almacenamiento en la nube.

Este módulo está compuesto por tres servicios especializados que trabajan de forma coordinada: el post-service, el media-service y el moderation-service. El post-service

gestiona el ciclo de vida completo de una publicación, desde su creación hasta su eliminación, utilizando transacciones explícitas de PostgreSQL con puntos de guardado (SAVEPOINT) que garantizan que si la inserción de hashtags falla, la publicación se crea igualmente sin etiquetas en lugar de fallar por completo. El media-service se encarga de recibir la imagen del usuario, subirla al bucket privado de Amazon S3 mediante el SDK de AWS y someterla a análisis de moderación automática con AWS Rekognition, rechazando el contenido inapropiado antes de que llegue a la base de datos. Finalmente, el moderation-service valida los hashtags de la publicación contra la lista de palabras prohibidas almacenada en la base de datos, actuando como filtro previo al COMMIT de la transacción. El estado inicial de toda publicación es PENDIENTE, transitando a PUBLICADO únicamente tras superar ambos filtros de moderación.

10.4 Módulo de Administración

Este proporciona un panel exclusivo para el rol de admin donde se gestionan los usuarios, se gestiona el contenido y el contenido de la tabla de palabras prohibidas o blacklist.

El acceso a cualquier endpoint de este módulo está protegido por el middleware RBAC del API Gateway, de forma que un usuario con rol USER que intente acceder recibirá un HTTP 403 tanto en el backend como en el frontend, que lo redirige automáticamente al feed. El panel centraliza cinco funcionalidades principales: el dashboard con métricas del sistema (publicaciones por día, likes por día y usuarios bloqueados), la moderación de contenido que permite aprobar, rechazar o bloquear publicaciones según su estado (PENDIENTE, PUBLICADO o BLOQUEADO), la gestión de cuentas de usuario mediante activación y desactivación sin necesidad de eliminar el registro, la administración de la lista de palabras prohibidas en formato JSON con confirmación obligatoria antes de aplicar cambios, y el log de auditoría completo y filtrable por fecha, usuario y tipo de acción, cuya consulta está restringida a nivel de base de datos mediante el rol admin_read_role con permisos de solo lectura sobre la tabla audit_logs.

10.5 Módulo de Búsqueda

Este módulo genera un feed principal ordenado descendentemente por los likes y modo explorar, además de implementar la búsqueda por texto libre y hashtags específicos.

El feed principal, servido por el post-service, retorna únicamente publicaciones con estado PUBLICADO ordenadas por el campo likes_count de forma descendente, utilizando la fecha de creación como criterio de desempate para publicaciones con igual número de votos. El modo Explorar ofrece un ranking global de las publicaciones más populares de toda la plataforma sin restricción temporal. El search-service implementa dos modalidades de búsqueda: la búsqueda por hashtag exacto, que realiza un JOIN entre las tablas post_hashtags y hashtags para filtrar publicaciones que contengan la etiqueta solicitada, y la búsqueda full-

text sobre el campo de descripción de las publicaciones utilizando la función to_tsvector de PostgreSQL con soporte para español, acompañada de un índice GIN que garantiza tiempos de respuesta eficientes incluso con grandes volúmenes de datos. Todos los resultados de este módulo están paginados con un límite de 10 publicaciones por solicitud y son de solo lectura, sin modificar ningún dato de la base de datos.

11. Diagrama de componentes

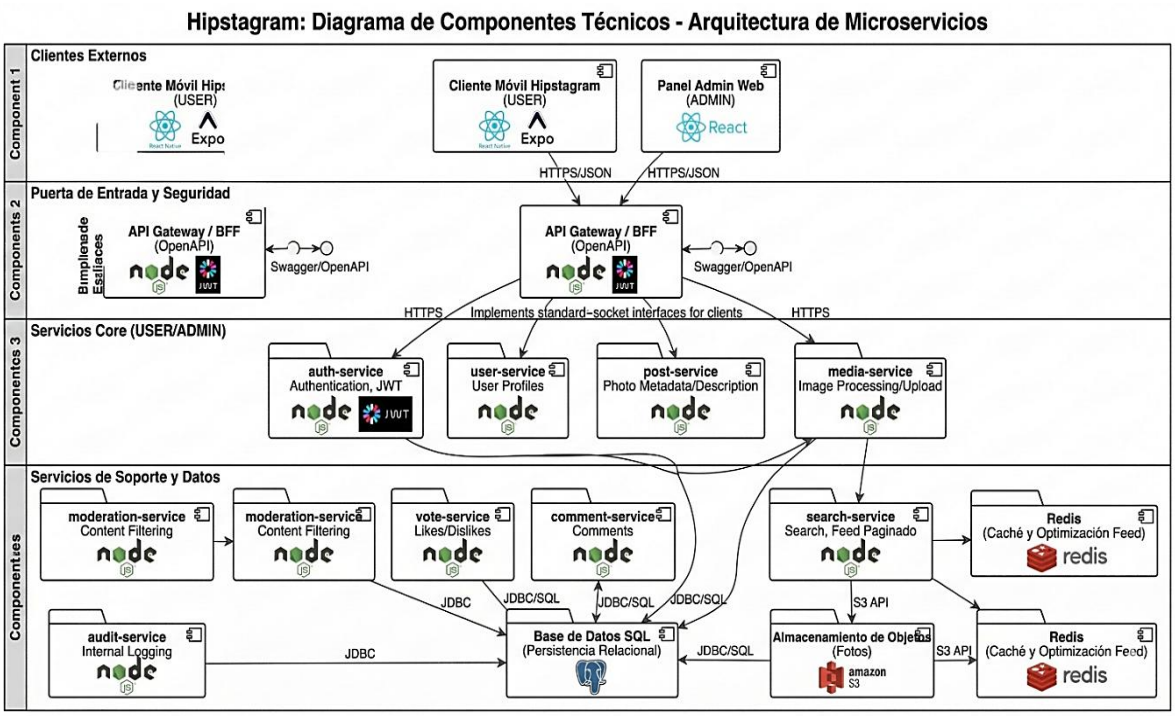
El diagrama presenta la arquitectura técnica completa del sistema Hipstagram, organizada en cuatro capas horizontales que reflejan la separación de responsabilidades del sistema.

Capa 1 — Clientes Externos: El sistema atiende tres tipos de clientes: dos instancias de la aplicación móvil construida con React Native y Expo para el rol USER, y un Panel de Administración Web desarrollado en React para el rol ADMIN. Todos se comunican exclusivamente mediante HTTPS/JSON hacia la capa de entrada.

Capa 2 — Puerta de Entrada y Seguridad: El API Gateway actúa como el único punto de entrada del sistema, implementado en Node.js con soporte JWT para autenticación y documentación mediante Swagger/OpenAPI. Ningún servicio interno es accesible directamente desde los clientes; toda solicitud pasa obligatoriamente por esta capa que valida identidad, rol y formato antes de enrutar.

Capa 3 — Servicios Core: Conformada por los cuatro microservicios de negocio principal: auth-service (autenticación y gestión de JWT), user-service (perfiles de usuario), post-service (metadatos y descripción de fotografías) y media-service (procesamiento y subida de imágenes). Cada servicio corre de forma independiente en Node.js y se comunica con las capas inferiores según su dominio.

Capa 4 — Servicios de Soporte y Datos: Concentra los servicios especializados — moderation-service (filtrado de contenido), vote-service (likes/dislikes), comment-service (comentarios), search-service (búsqueda y feed paginado) y audit-service (registro interno de eventos) — junto con las tres capas de persistencia: la Base de Datos SQL PostgreSQL accedida mediante JDBC/SQL, el almacenamiento de objetos Amazon S3 para las fotografías accedido vía S3 API, y Redis como capa de caché para optimización del feed.



12. Definición de requerimientos no funcionales

12.1 Seguridad.

Se implementara la seguridad desde el primer nivel, se implementara con autenticación de usuarios mediante roles para dar acceso a las funciones que son designadas desde la base de datos, además de implementar SP para la automatización de la base de datos.

12.2 Validación.

La validación de los datos desde los usuarios hasta los hashtags que estén ingresando los usuarios, así como el máximo de caracteres en la descripción de una publicación.

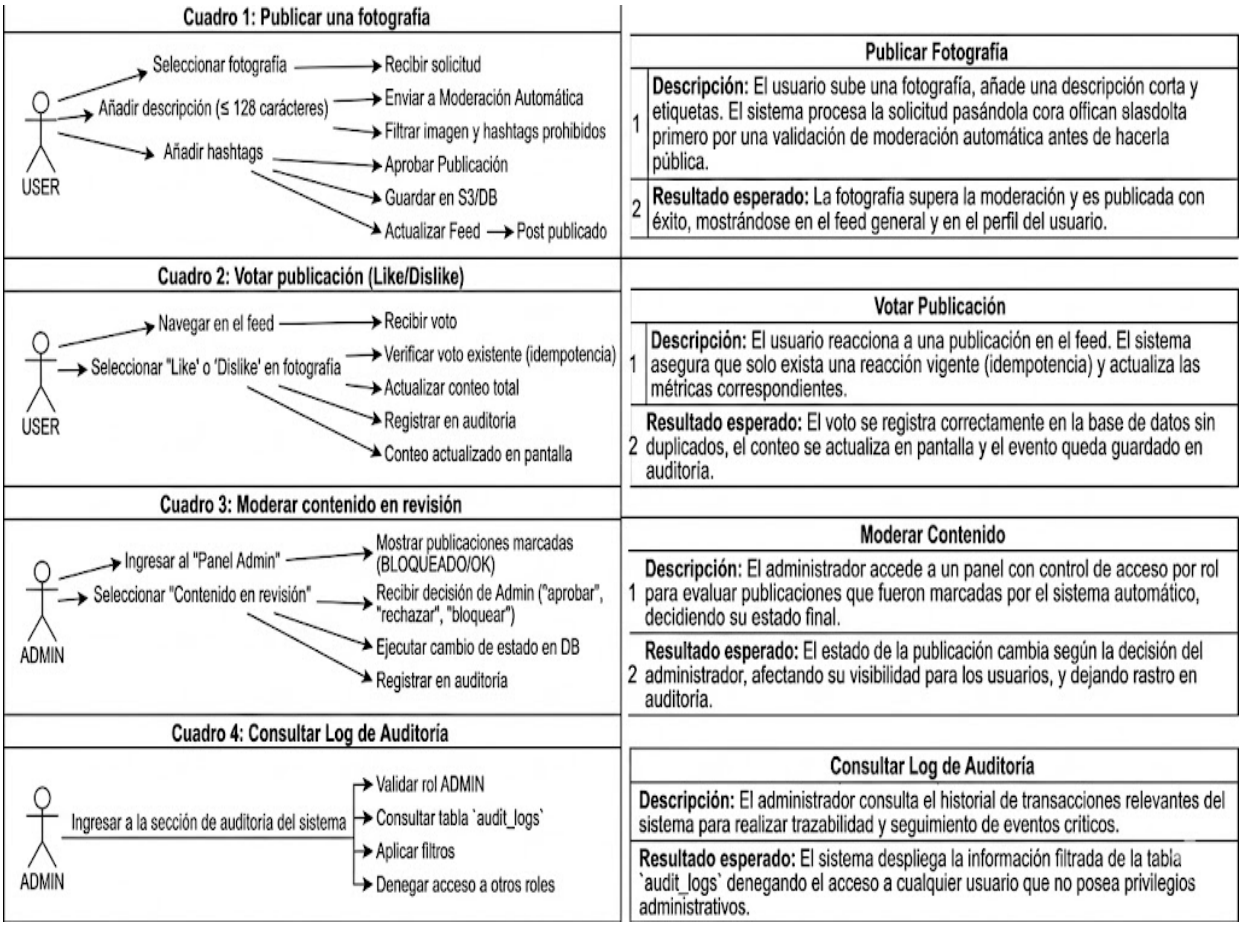
12.3 Rendimiento.

Se implementará la paginación en el feed, en los resultados de la búsqueda y en los comentarios, junto con un manejo eficiente de los likes.

12.4 Disponibilidad.

Soporte para el rendimiento, reinicio automático de los contenedores de Docker y configuración de healt check.

13. Diagramas de casos de uso



13.1 Descripción de los Diagramas de Flujo del Sistema

Los diagramas presentados describen los flujos de interacción más representativos del sistema Hipstagram, mostrando la secuencia de pasos desde la acción del usuario hasta la respuesta del sistema, junto con el resultado esperado en cada caso.

Cuadro 1 — Publicar una Fotografía (USER)

El usuario inicia el flujo seleccionando una fotografía, añadiendo una descripción de hasta 128 caracteres y etiquetando hashtags relevantes. El sistema recibe la solicitud y la somete inmediatamente a un proceso de moderación automática en dos niveles: análisis de la imagen mediante AWS Rekognition para detectar contenido inapropiado, y validación de los hashtags contra la lista de palabras prohibidas almacenada en la base de datos. Solo si ambas validaciones son superadas el sistema aprueba la publicación, la persiste en S3 y PostgreSQL, actualiza el feed general y marca el post como PUBLICADO. Si cualquiera de los filtros detecta contenido no permitido, la solicitud es rechazada antes del COMMIT y el usuario recibe un mensaje descriptivo del motivo.

Cuadro 2 — Votar Publicación Like/Dislike (USER)

El usuario navega por el feed y selecciona una reacción (Like o Dislike) sobre una fotografía. El sistema recibe el voto y aplica una verificación de idempotencia consultando si ya existe un voto previo del mismo usuario sobre esa publicación. Si existe, evalúa si la nueva reacción es igual a la anterior (sin cambio) o diferente (cambio de reacción), actualizando los contadores likes_count y dislikes_count de forma atómica dentro de una transacción. El resultado se refleja inmediatamente en la pantalla mediante actualización optimista del DOM, y el evento queda registrado en la tabla audit_logs para trazabilidad. La restricción UNIQUE(usuario_id, publicacion_id) en la base de datos actúa como segunda línea de defensa contra votos duplicados, independientemente de la lógica de aplicación.

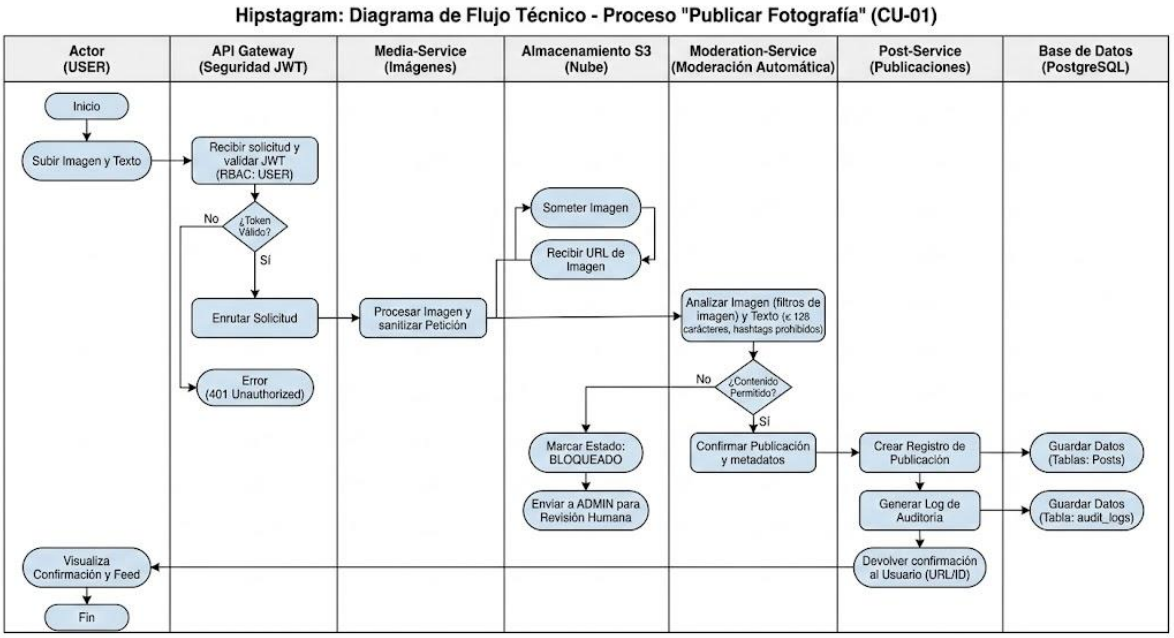
Cuadro 3 — Moderar Contenido en Revisión (ADMIN)

El administrador accede al panel exclusivo y navega a la sección de contenido en revisión, donde el sistema muestra únicamente las publicaciones con estado BLOQUEADO o PENDIENTE que requieren intervención manual. Tras revisar el contenido, el administrador emite una decisión: aprobar (cambia el estado a PUBLICADO y hace la publicación visible en el feed), rechazar (mantiene el estado como BLOQUEADO con razón registrada) o bloquear una publicación ya publicada que deba retirarse. Cada decisión ejecuta el cambio de estado correspondiente en la base de datos mediante una transacción explícita y registra el evento en audit_logs con el identificador del administrador, la acción ejecutada, el identificador de la publicación afectada y la dirección IP de origen. El acceso a este módulo está protegido en doble capa: el middleware RBAC del API Gateway valida el rol en el JWT, y el rol admin_read_role de PostgreSQL restringe las operaciones permitidas a nivel de base de datos.

Cuadro 4 — Consultar Log de Auditoría (ADMIN)

El administrador accede a la sección de auditoría del sistema, momento en que el backend valida que el token JWT contenga el rol ADMIN antes de ejecutar cualquier consulta. El sistema recupera los registros de la tabla audit_logs aplicando los filtros seleccionados por el administrador: rango de fechas, nombre de usuario o correo, y tipo de acción (registro, login, publicación, voto, comentario, moderación, entre otros). La consulta utiliza el rol admin_read_role de PostgreSQL, que tiene permisos exclusivamente de SELECT sobre esta tabla, garantizando que ningún proceso de aplicación pueda modificar o eliminar registros de auditoría, ni siquiera accidentalmente. Cualquier intento de acceso desde un usuario con rol USER es denegado con HTTP 403 tanto en el API Gateway como en el microservicio, y el intento queda a su vez registrado en el propio log de auditoría.

14. Diagramas de flujo de datos



El diagrama describe el flujo técnico completo del caso de uso más representativo del sistema: la publicación de una fotografía. Involucra seis componentes que trabajan en secuencia coordinada, cada uno con responsabilidad única dentro de la arquitectura de microservicios.

API Gateway — Validación de Identidad

Es el primer punto de contacto. Verifica criptográficamente el token JWT del usuario, comprueba que el rol sea USER mediante RBAC y, si el token es inválido o expirado, interrumpe el flujo retornando HTTP 401 sin invocar ningún servicio interno. Si es válido, enruta la solicitud al media-service con los headers de identidad adjuntos.

Media-Service y Amazon S3 — Procesamiento de Imagen

Valida el tipo y tamaño del archivo (máx. 5 MB), sube la imagen al bucket privado de S3 mediante el SDK de AWS autenticado con IAM, y retorna la URL del objeto para que el siguiente componente pueda referenciarla directamente, evitando transferir el binario entre servicios.

Moderation-Service — Moderación Automática

Analiza la imagen con AWS Rekognition y valida los hashtags contra la tabla banned_words. Si detecta contenido inapropiado en cualquiera de los dos niveles, marca la publicación como

BLOQUEADA y la deriva al panel de administración para revisión humana. Solo si ambos análisis son satisfactorios confirma la publicación al post-service.

Post-Service y PostgreSQL — Persistencia

Ejecuta una transacción explícita que inserta el registro en la tabla posts, asocia los hashtags mediante post_hashtags con un SAVEPOINT intermedio, y genera el registro de auditoría en audit_logs. Solo si todo es exitoso se ejecuta el COMMIT. Cualquier fallo desencadena un ROLLBACK que deja la base de datos en su estado anterior.

Confirmación al Usuario

El post-service retorna HTTP 201 al cliente con los datos de la publicación. La app muestra la pantalla "¡Publicación Exitosa!" y redirige al feed donde la fotografía ya es visible para todos los usuarios.

15. Prototipos e interfaces de los requerimientos funcionales

Los prototipos de interfaz son representaciones visuales del sistema que muestran cómo lucirá y funcionará cada pantalla antes de que sea programada. Actúan como el plano arquitectónico del frontend: el equipo de desarrollo los usa como guía exacta de qué mostrar, qué validar y cómo responder ante las acciones del usuario.

El diseño visual fue elaborado en Figma siguiendo una estética oscura con acentos en amarillo (#F5C518), coherente con la identidad visual de la plataforma Hipstagram. Los prototipos corresponden a una aplicación móvil .

En este documento se presenta el prototipo de baja fidelidad por cada requerimiento funcional del sistema, incluyendo: el diseño visual de la pantalla, los elementos de interfaz presentes, las validaciones que aplican en cada campo, los mensajes de error y éxito que el usuario verá, el flujo de navegación.

15.1 Prototipo en Figma

Enlace:
https://www.figma.com/proto/EAu8E9WjK8a4aC7VQ5XnRU/HIPSTAGRAM?node-id=2-416&p=f&t=FyOY4s9t1HWeKFgY-1&scaling=scale-down&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=2%3A416

15.2 Paleta de Colores del Sistema de Diseño

Color	Valor HEX	Uso
Amarillo Hipstagram	#F5C518	Botones CTA, acento principal, borde de pantallas activas, ícono de like activo.
Fondo oscuro	#1A1A1A	Fondo general de todas las pantallas de la app móvil.
Tarjeta / Card	#2A2A2A	Fondo de cards de publicaciones, modales y paneles.
Texto principal	#FFFFFF	Texto principal sobre fondo oscuro.
Texto secundario	#AAAAAA	Metadatos, timestamps, placeholders.
Error	#E53935 (Rojo)	Mensajes de error, botón Rechazar, dislike activo.
Exito	#43A047 (Verde)	Confirmaciones, botón Aprobar, estado PUBLICADO.

15.3 Mapa de Navegación del Sistema

RF	Pantalla	Rol	Navega hacia...
RF-01	Registro de usuario	Público	→ Feed principal
RF-02	Inicio de sesión	Público	→ Feed (RF-05) si USER / → Panel Admin (RF-14) si ADMIN
RF-03	Cerrar sesión	USER / ADMIN	→ Pantalla de Login (RF-02)
RF-04	Nueva publicación	USER	→ Feed (RF-05) tras publicar exitosamente
RF-05	Feed principal	USER / ADMIN	→ RF-10 (detalle), RF-04 (+), RF-11/12 (búsqueda), RF-13 (perfil)
RF-06	Explorar / Top Likes	USER / ADMIN	→ RF-10 (detalle de publicación)
RF-07	Eliminar publicación	USER (autor)	→ RF-05 — Feed Principal de Publicaciones
RF-08	Like / Dislike	USER	Inline en RF-05 y RF-10 — sin cambio de pantalla
RF-09	Comentar publicación	USER	Inline en RF-10 — sin cambio de pantalla

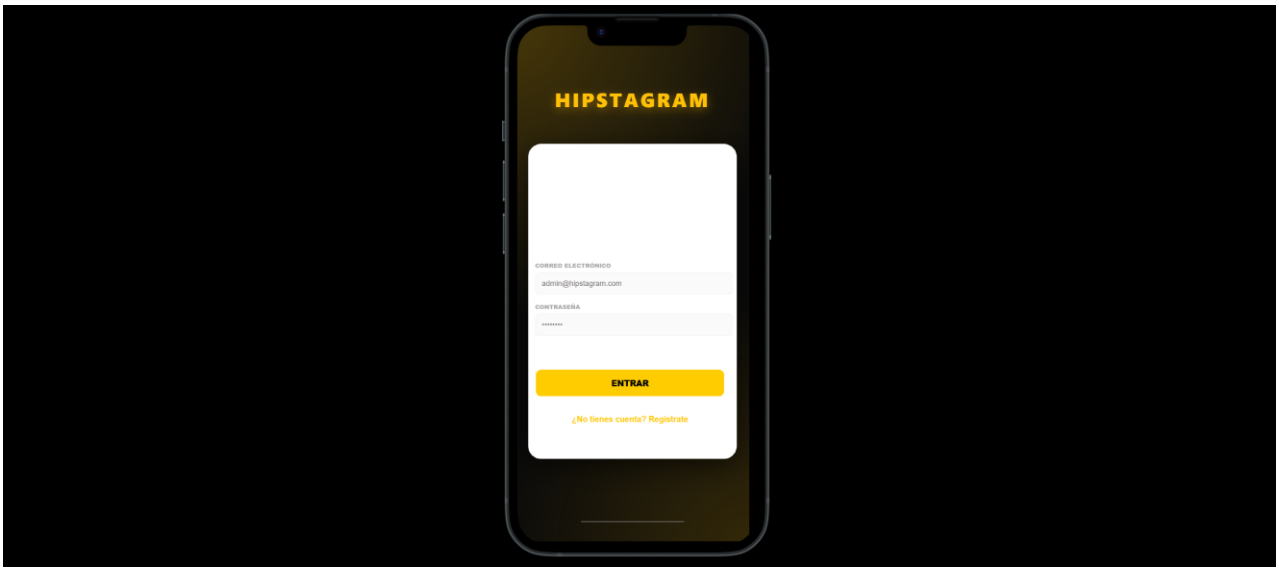
RF-10	Detalle de publicación	USER / ADMIN	→ RF-13 (perfil del **autor**), RF-11 (click en #hashtag)
RF-11	Búsqueda por hashtag	USER / ADMIN	→ RF-05 — Feed Principal de Publicaciones
RF-12	Búsqueda texto libre	USER / ADMIN	→ RF-05 — Feed Principal de Publicaciones
RF-13	Perfil de usuario	USER	→ RF-05 — Feed Principal de Publicaciones
RF-14	Panel de Administración	ADMIN	Módulos internos del panel: métricas, moderación, usuarios, palabras, auditoría

RF-01 — Registro de Usuario

ROL	**Público — sin autenticación requerida**
Descripción	El usuario nuevo crea una cuenta en Hipstagram proporcionando nombre de usuario, correo electrónico y contraseña.
Redirige a	Feed Principal (RF-05) tras registro exitoso

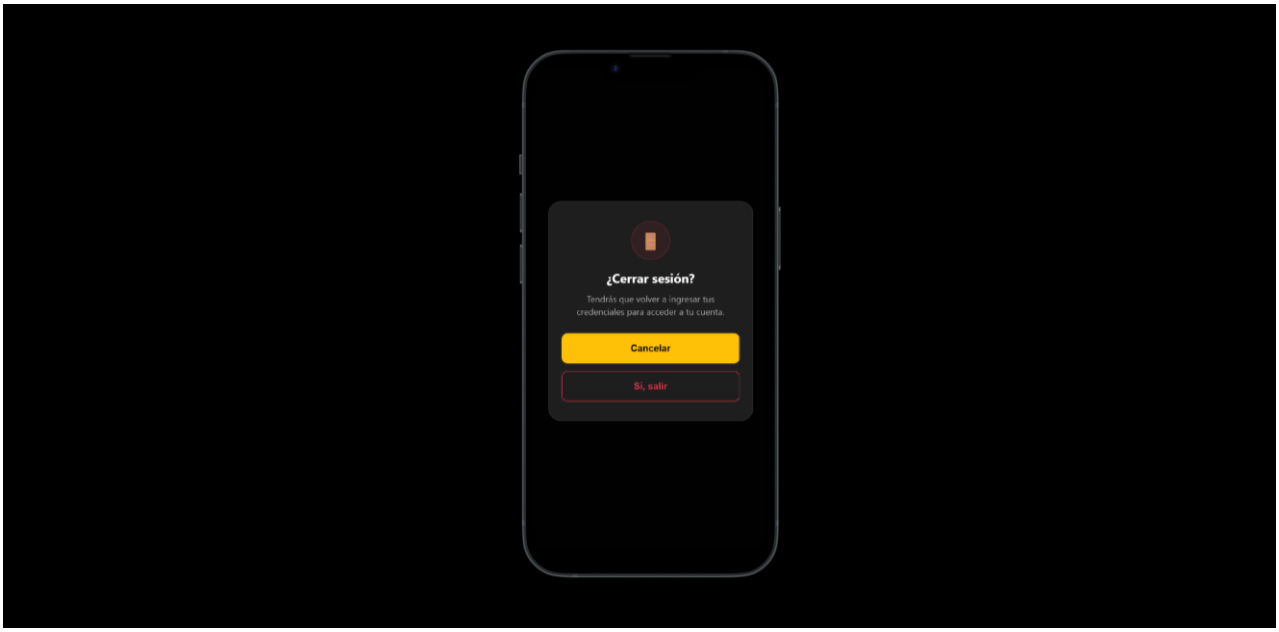
RF-02 — Inicio de Sesión (Login)

ROL	Público — sin autenticación requerida
Descripción	El usuario registrado se autentica con su correo y contraseña. El sistema verifica las credenciales.
Redirige a	Feed Principal (RF-05) si role='USER' / Panel Admin (RF-14) si role='ADMIN'



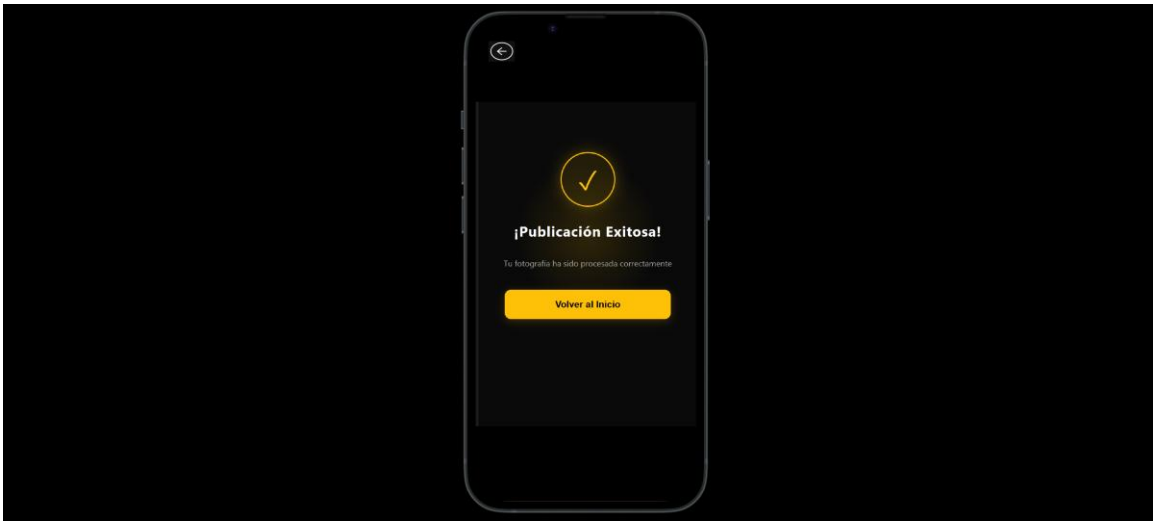
RF-03 — Cerrar Sesión (Logout)

ROL	USER / ADMIN — requiere estar autenticado
Descripción	El usuario autenticado cierra su sesión. El usuario es redirigido a Login.
Redirige a	Pantalla de Login (RF-02)



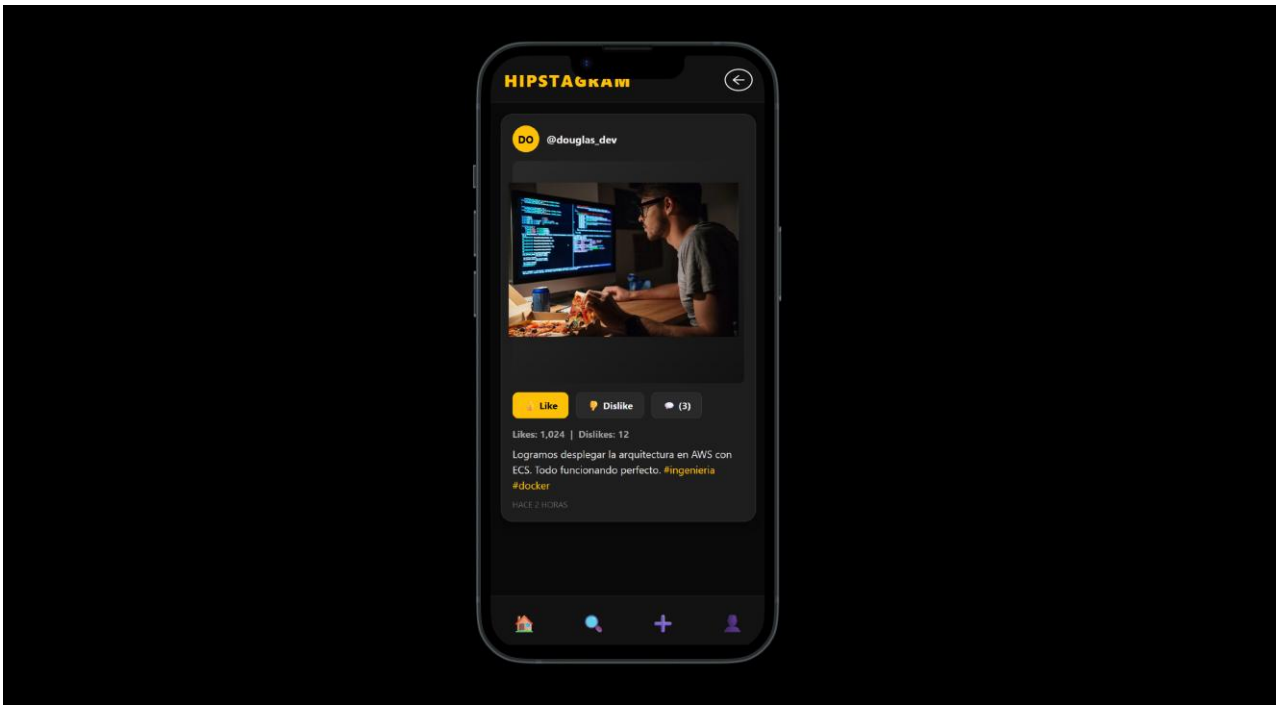
RF-04 — Crear Nueva Publicación

ROL	USER — requiere Access Token válido
Descripción	El usuario autenticado sube una foto, agrega descripción de hasta 128 caracteres y hashtags opcionales.
Redirige a	Pantalla de confirmación '¡Publicación Exitosa!' → Feed Principal (RF-05)



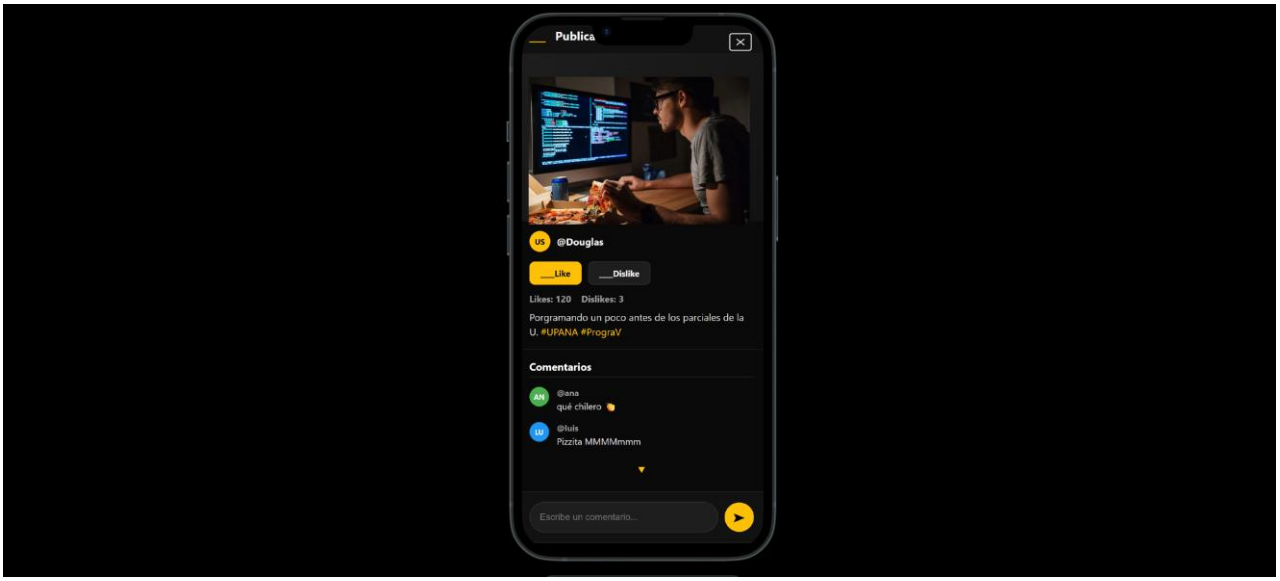
RF-05 — Feed Principal de Publicaciones

ROL	USER / ADMIN — requiere autenticación
Descripción	Muestra las publicaciones con estado PUBLICADO ordenadas por likes_count de forma descendente. Está paginada muestra las publicaciones por carga. El feed es la pantalla central de la app y punto de partida para todas las demás funciones.
Redirige a	RF-10 (detalle de post) / RF-04 (nueva publicación) / RF-11/12 (búsqueda) / RF-13 (perfil)



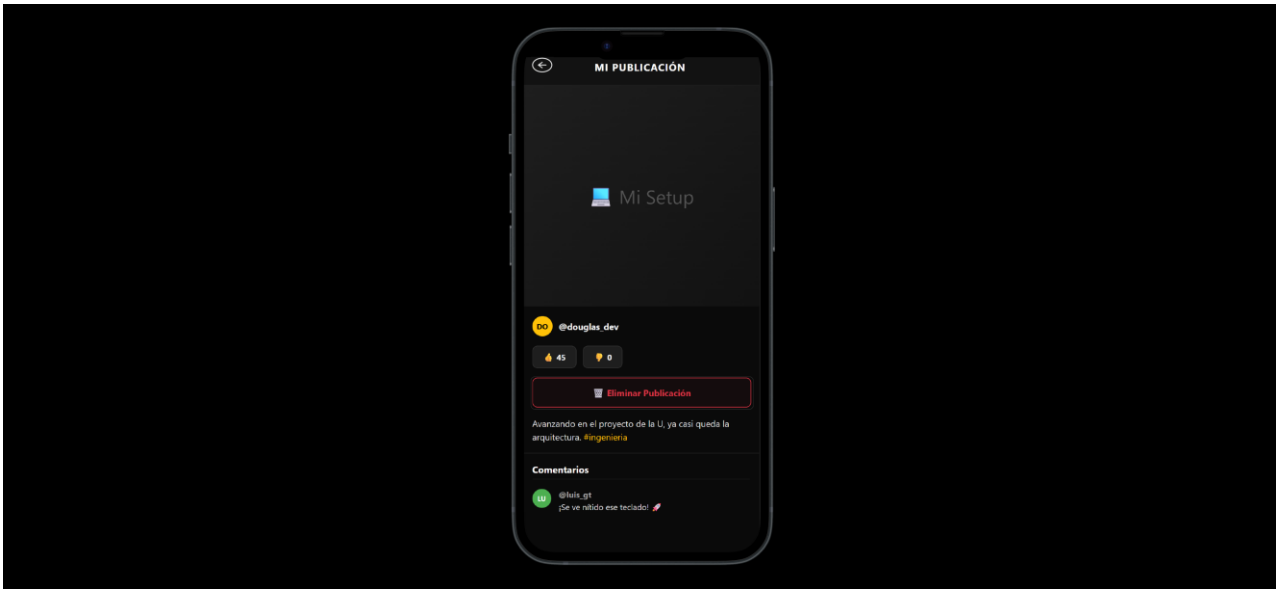
RF-06 — Explorar / Top Likes Global

ROL	USER / ADMIN — requiere autenticación
Descripción	Muestra detalles de una publicación en concreto. Muestra la cantidad de likes, dislikes y comentarios.
Redirige a	RF-10 (detalle de publicación seleccionada)



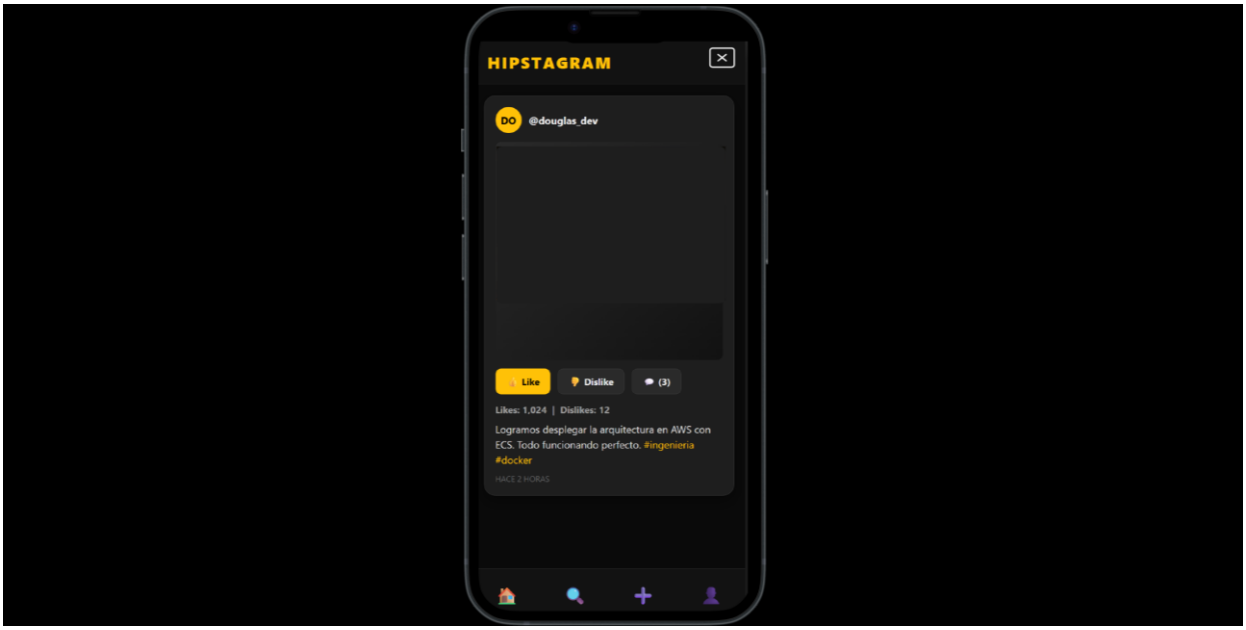
RF-07 — Eliminar Publicación Propia

ROL	USER (solo el autor de la publicación)
Descripción	El autor puede eliminar su propia publicación.
Redirige a	Feed Principal (RF-05) tras eliminar exitosamente



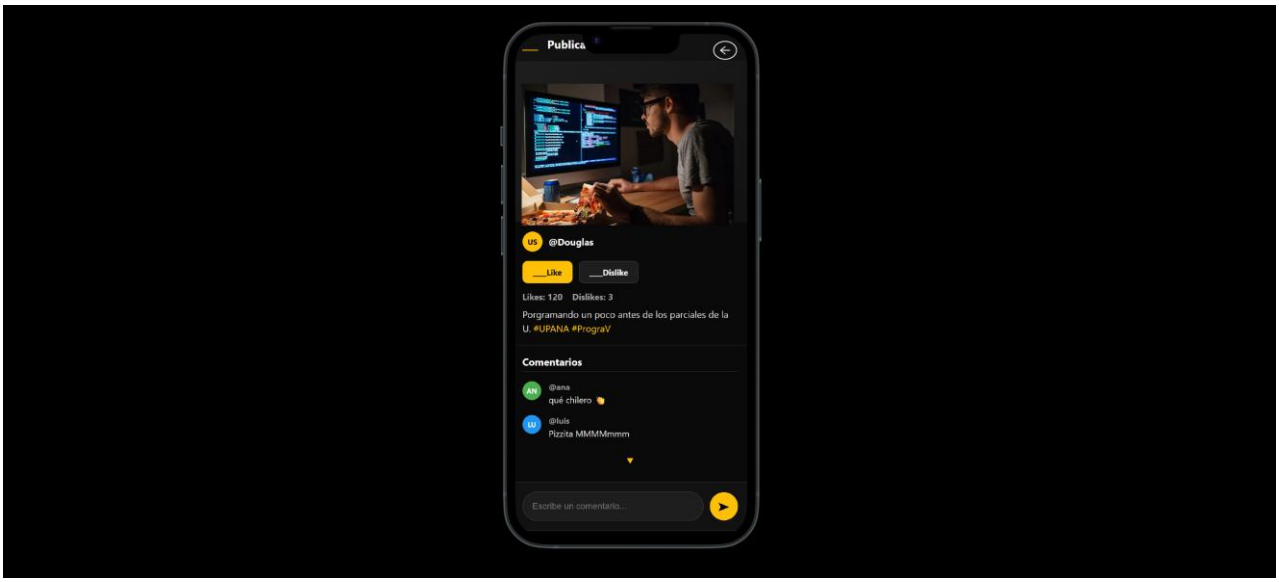
RF-08 — Like / Dislike (Sistema de Votación)

ROL	USER — requiere autenticación. No puede votar sus propias publicaciones.
Descripción	El usuario interactúa con publicaciones ajenas mediante like o dislike.
Redirige a	Sin cambio de pantalla — actualización inline en RF-05 y RF-10



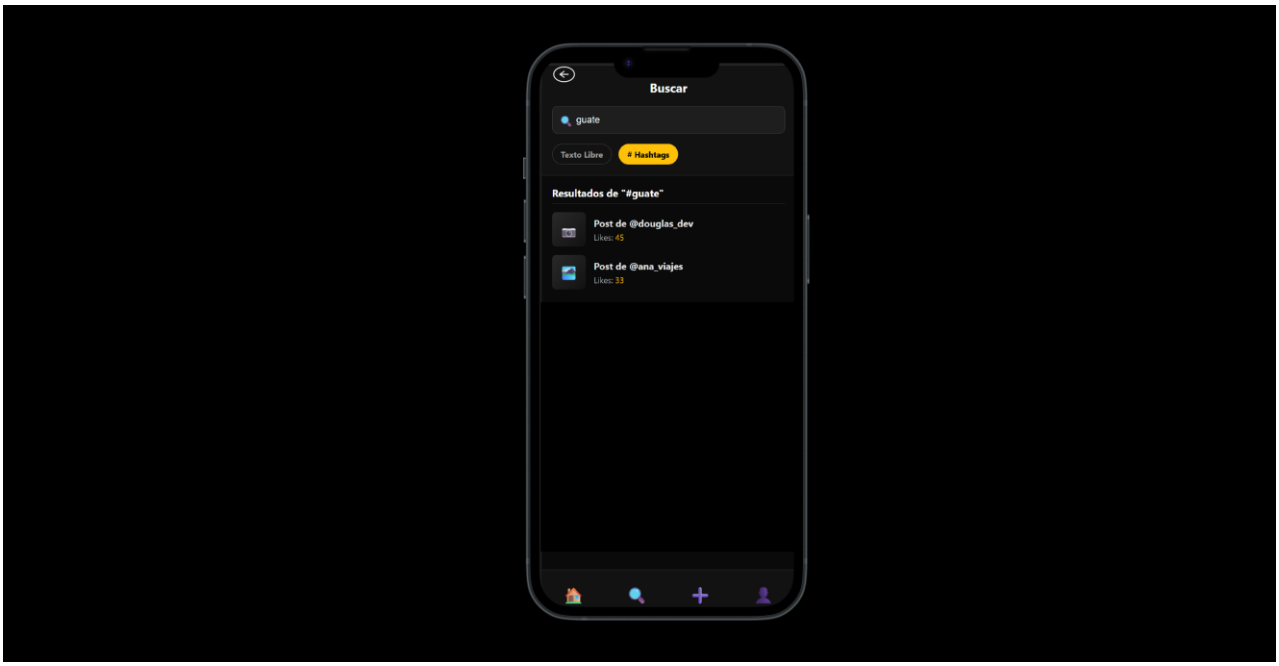
RF-09 / RF-10 — Comentarios y Detalle de Publicación

ROL	USER / ADMIN — requiere autenticación
Descripción	La pantalla de detalle muestra la publicación completa con imagen en tamaño grande, descripción, hashtags como links de búsqueda, contadores de votos, botones like/dislike y la sección de comentarios paginados, es la sección de comentarios dentro de la misma pantalla.
Redirige a	sin salir de la pantalla para comentar



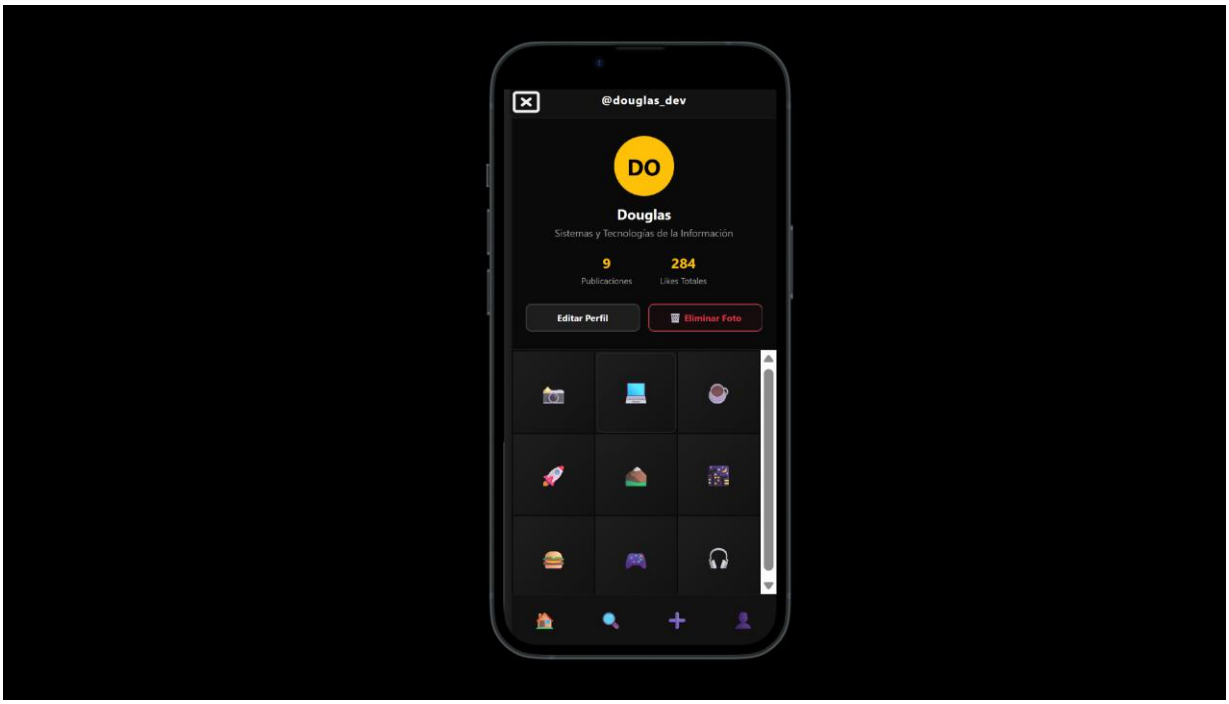
RF-11 / RF-12 — Búsqueda por Hashtag y Texto Libre

ROL	USER / ADMIN — requiere autenticación
Descripción	Permite buscar publicaciones filtrando por un hashtag específico (solo publicaciones PUBLICADAS con ese hashtag).
Redirige a	RF-05 — Feed Principal de Publicaciones



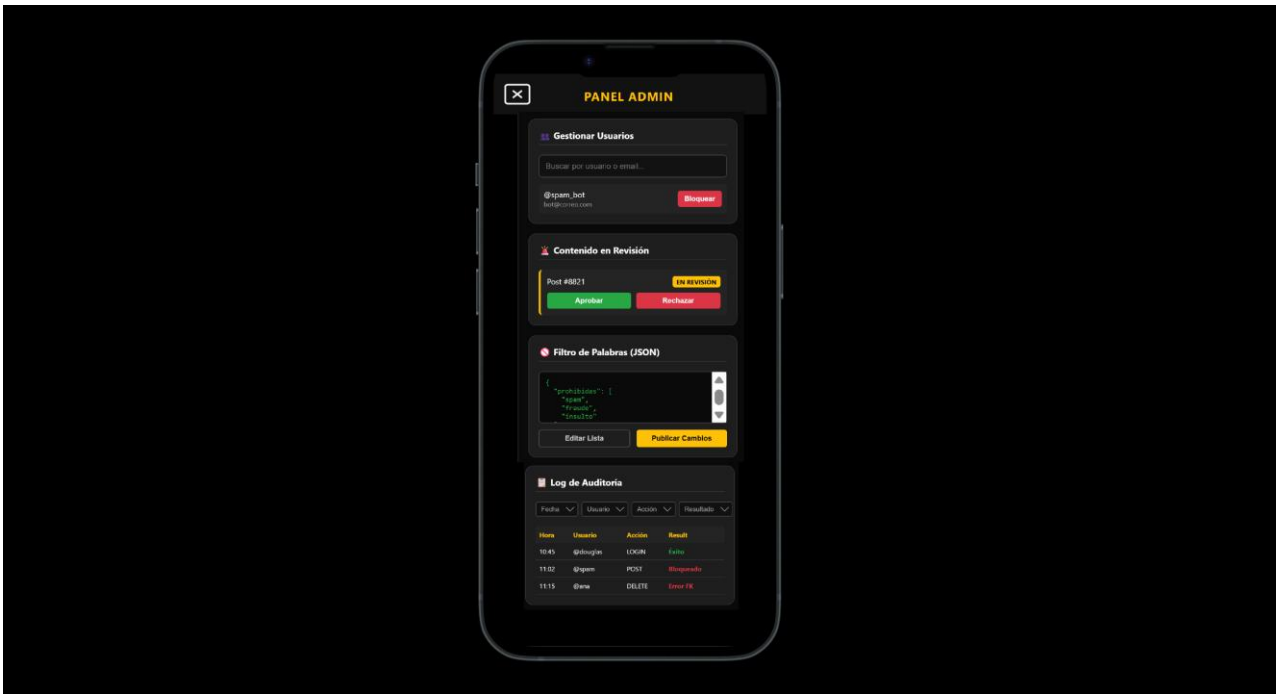
RF-13 — Ver Perfil de Usuario

ROL	USER — requiere autenticación
Descripción	El usuario puede ver su perfil propio con estadísticas (total de publicaciones, likes totales recibidos, total de comentarios) y su galería de publicaciones. También puede acceder al perfil público de otros usuarios. En el perfil propio aparece el botón 'Editar Perfil' y 'Eliminar Foto' (RF-07) en las publicaciones propias. En perfil ajeno, solo se muestra la información pública.
Endpoint	GET /users/me (perfil propio) | GET /users/:id (perfil ajeno)
Redirige a	RF-10 (detalle de publicación al hacer click en la galería)



Panel de Administración — RF-14

El panel de administración es una interfaz exclusiva para usuarios con rol ADMIN.



16. Recursos tecnológicos a utilizar (listado con costos)

16.1 Tecnologías de desarrollo.

16.1.1. Node.Js: Será la base para el desarrollo del proyecto.

16.1.2. React: Desarrollo de la aplicación para el rol de usuario

16.1.3. OpenApi: Estándar para definir los contratos de las Apis rest de los microcontroladores antes de programar.

16.2 Tecnologías de persistencia y almacenamiento.

16.1.4. PostgresSQL: Servirá como base de datos funcional como motor de SQL principal para la persistencia estructurada de usuarios, post, votos, comentarios.

16.1.5. AWS: Almacenamiento de objetos, buckets en la nube para el almacenamiento persistente y seguro de las fotografías subidas por los usuarios.

16.1.6. Redis: Caché en memoria para optimizar el rendimiento de búsqueda frecuentes del feed ordenado por likes

16.3 Tecnologías de infraestructura.

16.1.7. Git/Gitflow: Servirá para el control de versiones que es un estándar para el versionado de código y gestión de ramas.

16.1.8. Docker: Creación de imágenes para cada microservicio y orquestar localmente las pruebas de integración.

16.1.9. Jenkins: Automatización de pipeline de construcción, pruebas unitarias y despliegue continuo.

17. Prioridad de los requerimientos a desarrollar

17.1. Alta prioridad.

17.1.1. Implementación de los API Gateway con control de acceso basado en roles.
17.1.2. Desarrollo de servicios de media para subir fotografías.
17.1.3. Desarrollo del servicio para la gestión de perfiles.

17.2. Prioridad media.

17.1.4. Desarrollo de la implementación de la moderación automática.
17.1.5. Creación de la moderación automática antes de la publicación.

17.3 Baja prioridad.

17.1.6. Desarrollo del servicio para likes y comentarios.
17.1.7. Implementación del servicio para la búsqueda de perfiles y filtrado por hashtags.
17.1.8. Algoritmos para el feed y el modo de exploración.